

承诺书

我们完全清楚，在竞赛开始后参赛队员不能以任何方式，包括电话、电子邮件、“贴吧”、QQ 群、微信群等，与队外的任何人（包括指导教师）交流、讨论与赛题有关的问题；无论主动参与讨论还是被动接收讨论信息都是严重违反竞赛纪律的行为。

我们以中国大学生名誉和诚信郑重承诺，严格遵守竞赛章程和参赛规则，以保证竞赛的公正、公平性。如有违反竞赛章程和参赛规则的行为，我们将受到严肃处理。

我们授权全国大学生数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

(指导教师签名意味着对参赛队的行为和论文的真实性负责)

日期: 2026 年 08 月 28 日

(请勿改动此页内容和格式。此承诺书打印签名后作为纸质论文的封面，注意电子版论文中不得出现此页。以上内容请仔细核对，如填写错误，论文可能被取消评奖资格。)

赛区评阅编号：
(由赛区填写)

全国评阅编号：
(全国组委会填写)

2026 高教社杯全国大学生数学建模竞赛

编 号 专 用 页

赛区评阅记录（可供赛区评阅时使用）：

评阅人						
备注						

送全国评阅统一编号：

(赛区组委会填写)

(请勿改动此页内容和格式。此编号专用页仅供赛区和全国评阅使用，参赛队打印后装订到纸质论文的第二页上。注意电子版论文中不得出现此页。)

面向需求随机波动的 生鲜蔬菜补货与定价策略模型

摘要

生鲜蔬菜保鲜期短，需求在日期、星期和品类之间呈现明显随机波动，商超还面临进价事前未知和销售空间有限等问题。如何依据历史流水刻画随机需求，在缺货与损耗之间取得平衡，并协同制定每日补货与定价策略，是门店经营中的实际难点。为此，本文构建“数据审计—时间预测—价格响应—随机情景—整数优化”的分层策略模型。

针对问题一，采用 HAC 修正的描述统计、滞后相关及控制趋势、星期和月份后的残差相关，并对相关检验实施 FDR 校正。六类日销量均呈右偏长尾，滞后 1 日相关系数为 0.606–0.759，滞后 7 日仍为 0.476–0.564，均显著且达到中等以上效应。控制共同时间因素后，15 组品类组合中有 10 组满足 $q < 0.05$ 且 $|\rho| \geq 0.30$ ；低活跃单品的 2.5 kg 最低陈列量相当于约 4.12 个平均销售日，补货风险明显较高。

针对问题二，在对数尺度上建立含趋势、星期、年周期 Fourier 项及 1–7、14 日滞后的动态回归，并在动态回归和同周几中位数基线之间逐品类滚动选模，最终需求预测平均 WAPE 为 22.6%。价格端采用单品和日期双向固定效应回归，再以 7 日移动块抽样构造 600 个需求—成本—价格响应联合随机情景。将“已满足的基准需求”定义为实际销量与调价前需求的较小值，并以最低品类覆盖率和期望利润为双重评价指标构造 Pareto 前沿。前沿折中点对应最低覆盖率 86.00%，未来一周补货 2826.95 kg，期望利润 4770.83 元，较朴素规则提高 57.64%，最坏 10% 场景平均利润为 2630.04 元。

针对问题三，以近一周有正常销售的 49 个单品为候选集，用近期销量与长期份额进行交叉验证的收缩估计。在 27–33 个单品、单品补货量不低于 2.5 kg 且每个单品只选一种价格—补货方案的约束下，先最大化最低品类覆盖率，再保持该水平最大化期望利润，同时确定单品、补货量和售价。模型选取 33 个单品，补货 636.52 kg，最低覆盖率 93.65%，期望利润 427.41 元。

针对问题四，从需求识别、供应约束、损耗机制和价格内生性四个方面提出数据补充方案，优先采集实时库存与缺货、订货到货与闭店剩余、决策时供应报价及促销陈列记录。上述数据可将销量截尾、随机提前期、动态损耗和商品替代关系纳入多周期模型。

本文的特点在于：以滚动样本外表现逐品类选择预测模型和份额收缩参数；将需求、成本和价格响应的不确定性放入同一随机情景；以 Pareto 前沿刻画问题二中利润与覆盖率的权衡，并依据题意以词典序目标处理问题三的覆盖与利润顺序。约束回代和 SKU 数量前沿进一步给出利润、覆盖与尾部风险之间的边界。模型面向未来 7 日并随新数据滚动重估，不直接外推为长期经营规律。

关键字： 动态回归 双向固定效应 随机优化 混合整数规划

一、问题重述

1.1 问题背景

生鲜蔬菜是居民高频购买的日常商品，其供应是否稳定直接影响消费者的购物体验；但蔬菜又具有保鲜期短、质量衰减快和隔日残值低等特点。门店补货不足会造成缺货和顾客流失，补货过量则会增加折价销售与报损。因而，利用有限销售空间维持基本品类供给，同时控制损耗并改善经营收益，既具有直接的门店管理价值，也有助于减少生鲜食品浪费。

与耐用品不同，蔬菜门店通常需要在每日营业前完成订货和定价，而实际客流、消费者偏好、天气及临时促销会使当天需求随机波动；采购价格和可供量也可能在决策时尚未完全确定。售价既影响销量，又常由成本、鲜度和清仓需要共同决定，补货量、售价与最终损耗由此形成相互制约的策略组合。国内相关研究已从生鲜品订货定价、库存补货和货架配置等角度说明联合制定经营策略的必要性 [1, 2, 3]，数据驱动方法也为复杂环境下的库存控制与动态定价提供了新的实现途径 [4]。

实际建模还受到数据可见性的限制。收银流水记录的是已经成交的销量，而非完全可观测的潜在需求；当商品缺货、未上架或提前售罄时，低销量并不等于低需求。同时，品类层策略关系到整体供给稳定，单品层策略又受到货架容量、最低陈列量和长尾销量的约束。因此，需要先识别销量的分布与时间规律，再把需求预测误差、成本误差和价格响应不确定性传递到补货与定价策略中，而不能将一次点预测直接当作确定需求。

本文使用 2020 年 7 月 1 日至 2023 年 6 月 30 日 6 个蔬菜品类、251 个单品的销售流水和批发价格，以及近期损耗率。研究首先分析品类和单品销量规律，再制定 2023 年 7 月 1-7 日的品类补货与定价策略，并在销售空间约束下细化到 7 月 1 日的单品组合。其意义在于形成一套可由门店每日更新的短期滚动方法，把分散的交易记录转化为能够解释、检验和执行的经营策略。图1概括了随机需求下的门店经营闭环。

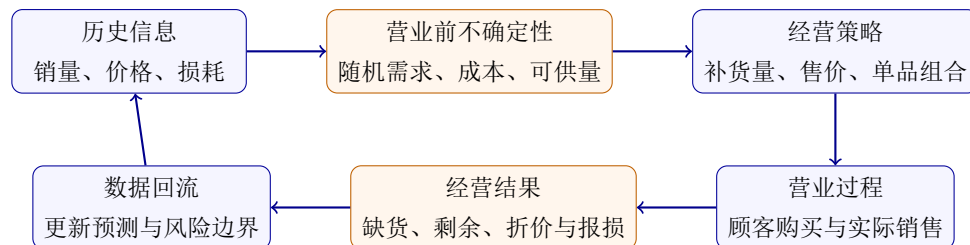


图 1 随机需求下生鲜蔬菜门店的经营策略闭环

1.2 问题分析

针对问题一，首先需要明确销量的统计口径。销售流水中同时包含正常销售和退货记录，因此本文按“日期—单品”汇总净销量，再依据单品所属品类得到品类日销量。在此基础上，分别从品类和单品两个层面分析销量分布。品类层面主要考察日销量的集中趋势、离散程度、偏态特征、周期性和滞后相关性；单品层面则关注销售活跃度、销量差异及长尾现象。对于品类之间的关系，除计算原始相关系数外，还需要控制趋势、星期和月份等共同时间因素，以避免将季节性变化误判为品类之间的稳定关联。

针对问题二，需要给出 2023 年 7 月 1 日至 7 日各蔬菜品类的日补货量和定价策略。首先根据历史日销量和批发价序列分别预测未来一周需求与成本，再利用单品历史售价、成本、折扣和销量估计条件价格响应。由于三类输入均存在误差，需要在联合情景下计算不同补货—价格组合的利润。题目没有给定服务水平，因此不能直接指定统一覆盖阈值；本文分别求解不同最低覆盖水平下的最大利润，以效率前沿呈现利润和供给保障的权衡，再从中选择算法折中方案。

针对问题三，需要将问题二的品类层决策落实到具体单品。由于单品序列稀疏，本文用近期销量和长期份额的收缩估计分配品类需求，并通过滚动验证选择收缩强度。题目中的“尽量满足各品类需求”具有明确优先顺序，因此先在 27–33 个单品和最低 2.5 kg 陈列量约束下最大化最低品类覆盖率，再保持该最优覆盖水平并最大化利润，得到 7 月 1 日的单品组合、补货量和售价。

针对问题四，需要结合前三问中实际遇到的数据限制提出补充建议。现有数据能够反映成交销量、批发价格和近期损耗率，但缺少逐时库存、缺货时点、订货与到货、闭店报损、促销类型及商品陈列等信息。因此，无法准确区分“没有需求”和“因缺货未能成交”，也难以完整识别供应波动、动态损耗和促销活动对销量的影响。为提高需求预测和补货决策的可靠性，应优先补充库存流转、订货到货、缺货和报损数据，并进一步记录促销、商品质量、陈列位置、天气节假日及客流等辅助变量。图2给出了各问题所使用的数据、分析方法和决策输出。

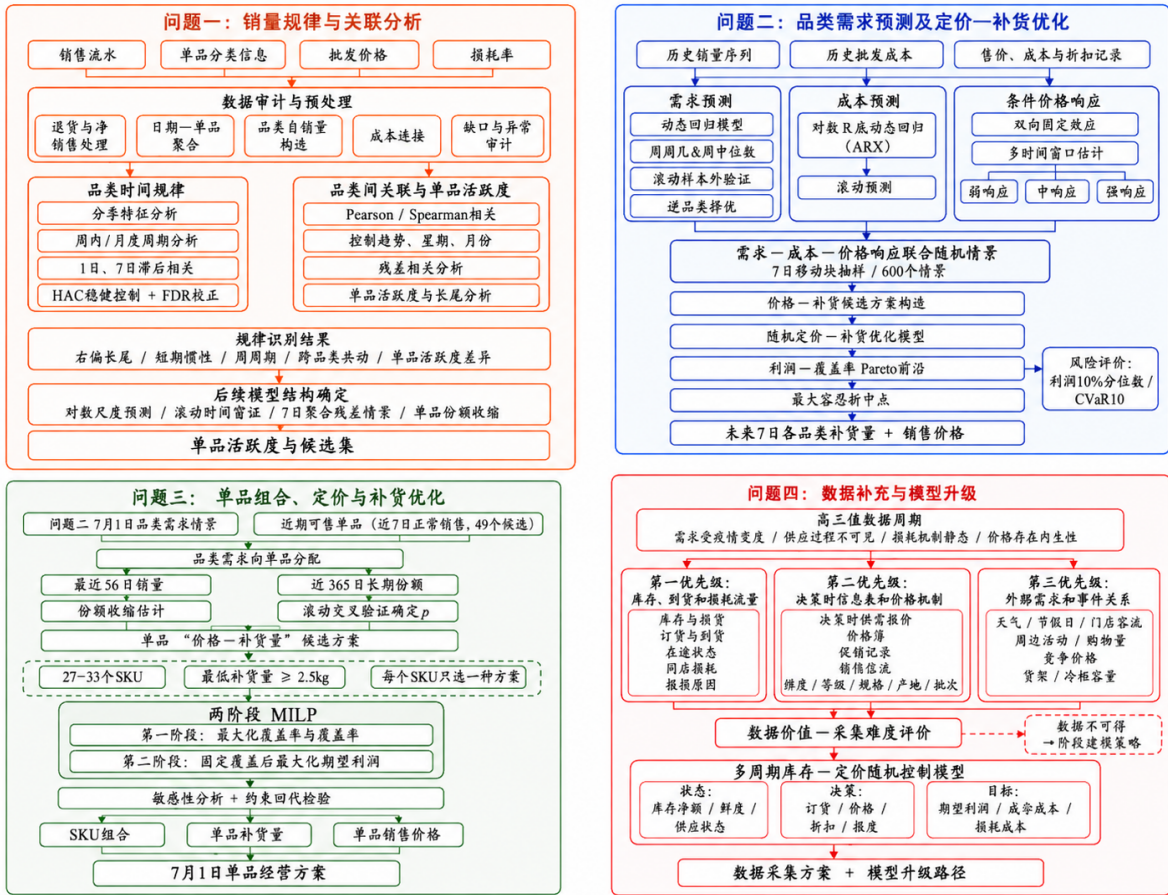


图2 本文总体研究流程

二、模型假设

结合题面信息与数据可识别范围，作如下假设。

- A1.** 在预测周内，商超营业制度、销售空间和基本定价机制与历史末期一致。该假设支持短期外推，但不支持长期制度变化后的预测。
- A2.** 附件中的销售和退货数量真实记录已实现交易；无流水日期或品类日含义未知，不能直接补为零。观测销量可能因缺货而低于潜在需求。
- A3.** 同一品类内，销售数量加权批发成本能够近似表示品类进货成本水平；单品未来成本按品类预测成本相对最近观测成本的比例调整。
- A4.** 附件4的近期损耗率可用于预测周，且损耗主要作用于补货量，使可售供给为 $(1 - \ell)Q$ 。该假设仅用于未来7日的短期近似，不解释长期鲜度演化。
- A5.** 条件价格响应系数只刻画历史经营制度下控制单品和日期效应后的关联，不视为严格因果弹性；优化仅在历史加成率支持域内搜索。
- A6.** 28个连续滚动验证日的联合残差能够近似代表预测周短期冲击；采用7日移动块抽样保留周内持续性和跨品类共振。

A7. 问题三中，2023 年 6 月 24-30 日存在正常销售的单品构成 7 月 1 日的可售候选集合；未选单品的潜在需求仍计入品类覆盖率分母。

这些假设分别对应短期稳定性、销量截尾、成本聚合、损耗外推、价格内生性、情景代表性和候选集合七类风险。后续检验与问题四的数据建议均围绕这些边界展开。

三、符号说明

本文模型涉及的主要索引、随机变量、决策变量及评价指标如表1所示。

表 1 主要符号及含义

符号	含义	符号	含义
i, c, t	单品、品类和自然日索引	s	联合不确定情景索引
$Y_{i,t}, Y_{c,t}$	单品日、品类日净销量/kg	$D_{c,t,s}^0$	不随本次定价缩减的基准需求
$D_{c,t,s}(m)$	加成率 m 下的情景需求	$Q_{c,t}$	品类补货量/kg
$S_{c,t,s}$	情景实际销售量/kg	$W_{c,t,s}$	情景批发成本/(元/kg)
$P_{c,t}$	建议销售价格/(元/kg)	ℓ_c, ℓ_i	品类或单品损耗率
$m_{c,t}, m_i$	成本加成率	β_c	条件价格响应系数
$x_{c,t,k}$	是否选择品类决策选项 k	z_i	单品 i 是否入选
$y_{i,h}$	是否选择单品决策选项 h	λ	最低品类基准需求覆盖率

四、数据预处理与审计

4.1 观测单位与数据连接

附件 1 含 251 个单品和 6 个品类；附件 2 含 878503 条扫码流水，其中 878042 条销售和 461 条退货；附件 3 含 55982 条日期—单品批发价；附件 4 含 251 个单品的近期损耗率。销售表先通过单品编码连接主数据，再以“日期+单品编码”精确连接同日成本，最后连接损耗率。46,599 个有流水单品日均能获得同日成本。

将原始交易量 q_r 按销售类型保留符号，定义

$$Y_{i,t} = \sum_{r \in (i,t)} q_r, \quad Y_{c,t} = \sum_{i: g(i)=c} Y_{i,t}. \quad (1)$$

退货记录本身为负值，因此不重复翻转。品类价格、成本、加成率和折扣占比均按当日正向销售数量加权，以避免不同单品规格造成无权平均失真。

4.2 缺失、异常与清洗原则

观测期共有 1095 个自然日，10 日无任何流水；在门店存在其他品类流水时，仍有 36 个日期—品类组合无记录。没有库存、上架与营业状态，故这些缺口保留为未知，不

将其解释为零需求。四张原始表不存在物理缺失、完整行重复和非法损耗率。

销售流水中 12651 条售价低于同日批发价，占正向销售的 1.44%；单笔销量最大 160 kg，对应鲜粽叶；加成率最大值受极低批发价影响而异常放大。上述记录均可能来源于折扣、节庆团购或真实经营，因此不按箱线图规则机械删除，而是保留并设置标志。清洗只规范字段类型、生成销售/退货/净销量指标并连接成本，未插补、缩尾或改写原始记录。该策略减少人为改变尾部和节庆信号的风险。

五、 问题一：销量分布规律及相互关系

5.1 品类销量的边际分布

对每个品类计算有效日数、均值、中位数、四分位数、偏度及变异系数。由于销量具有显著时间依赖，均值区间使用最大 28 阶滞后的 Newey–West 异方差自相关稳健协方差 [5]，不把每日观测误当独立样本。表2显示六类销量均右偏，花叶类日均销量 182.97 kg，规模最大；茄类仅 21.36 kg。水生根茎类的变异系数 0.84，为六类最高，表明仅用均值制定补货会低估风险。

表 2 六类蔬菜日净销量的分布统计

品类	有效日数	均值/kg	中位数/kg	四分位区间/kg	偏度	变异系数
花叶类	1085	182.97	173.19	[128.70, 223.51]	2.90	0.47
花菜类	1084	38.53	34.07	[23.31, 48.55]	1.53	0.59
水生根茎类	1085	37.40	30.19	[15.05, 50.82]	2.51	0.84
茄类	1050	21.36	18.87	[12.23, 27.13]	1.74	0.62
辣椒类	1085	84.41	72.92	[51.24, 102.13]	3.15	0.63
食用菌	1085	70.13	57.53	[39.26, 89.19]	3.00	0.69

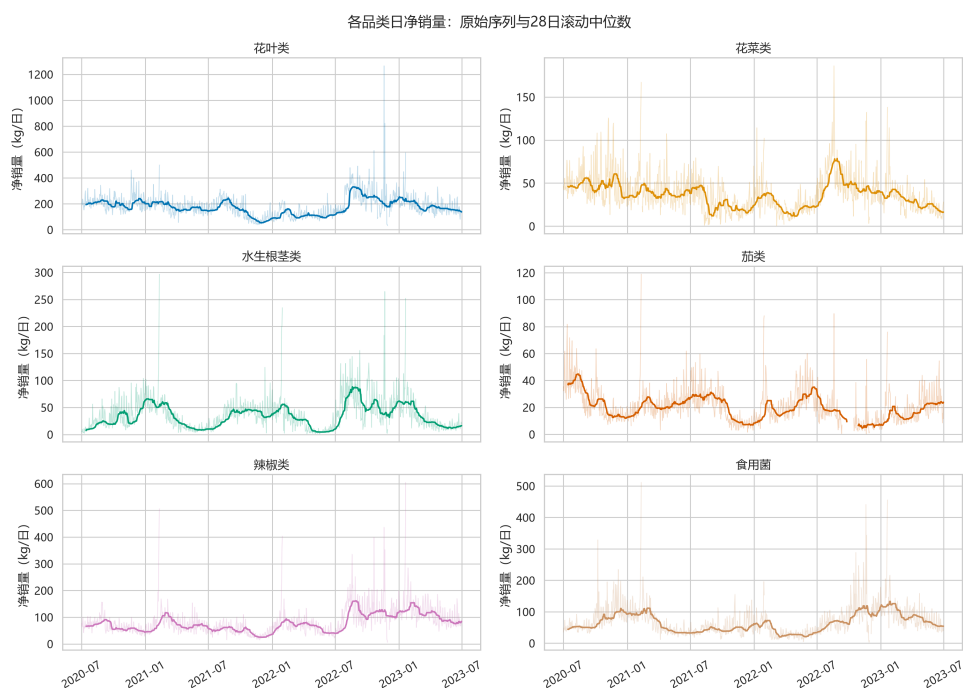


图3 六品类日净销量时间序列

图3表明销量存在阶段性水平变化、周内周期和少量尖峰。图4进一步显示，均值普遍大于中位数，极端日主要位于右尾。因此后续预测在对数尺度建模，并保留同周几中位数作为不易受尖峰影响的简单基线。

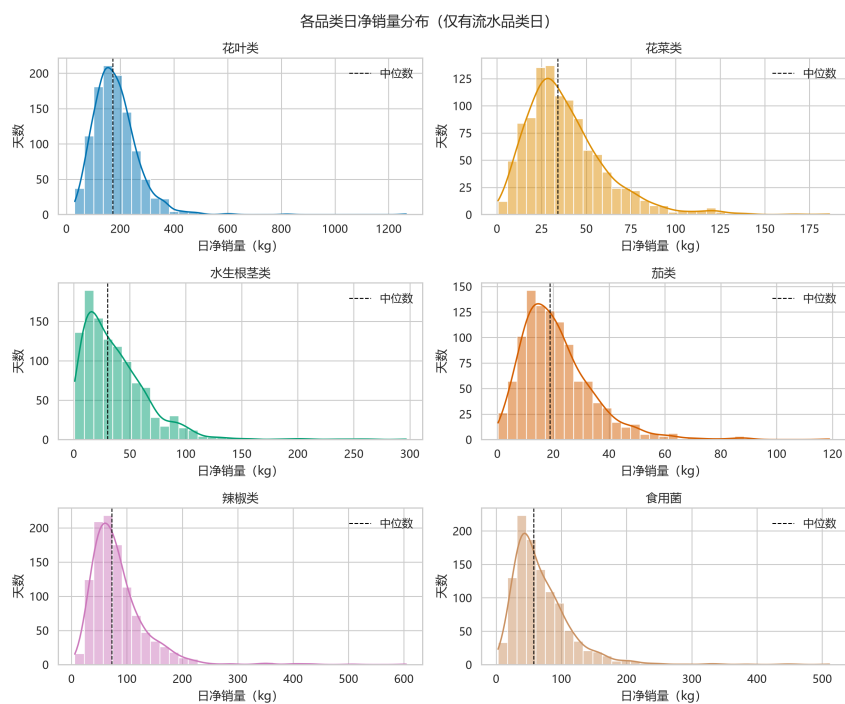


图4 六品类日净销量分布

5.2 周内、月度与动态依赖

六类蔬菜的周末中位销量整体高于工作日，但月度结构因品类而异：水生根茎类在年初较高，茄类在夏季较高，食用菌在年初和年末较高。这说明“星期”和“年周期”应同时进入预测模型，单一线性趋势不足以解释季节变化。

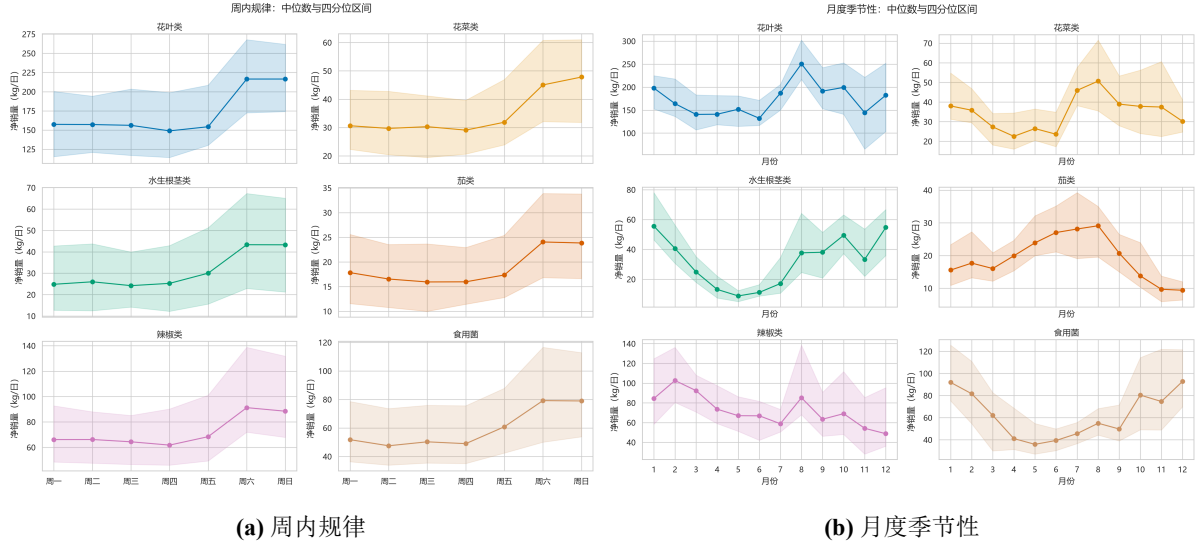


图5 品类销量的日历规律

如图5所示，周末效应在六个品类中方向较一致，但月度峰值并不同步：水生根茎类偏向年初，茄类在夏季抬升，食用菌在年初和年末较高。这种“共同星期效应与错位季节效应并存”的结构说明，后续相关分析必须先控制日历因素，预测模型也需同时包含星期项和年周期项。

对缺失日期不补零，仅使用当前与滞后观测均有效的日期对，计算

$$\hat{\rho}_c(k) = \frac{\sum_t (Y_{c,t} - \bar{Y}_c)(Y_{c,t-k} - \bar{Y}_c)}{\sum_t (Y_{c,t} - \bar{Y}_c)^2}. \quad (2)$$

为考虑相关系数自身的时间依赖，将当前值和滞后值分别标准化，以标准化滞后值解释当前值，并使用最大 28 阶 Newey–West 协方差估计系数区间。24 个“品类 × 滞后阶数”检验的 p 值统一进行 Benjamini–Hochberg 校正 [6]。本文预先规定： $q < 0.05$ 表示关联有统计支持；在此基础上， $|\rho| < 0.30$ 、 $0.30 \leq |\rho| < 0.50$ 和 $|\rho| \geq 0.50$ 分别记为弱、中等和强效应，只有显著且达到中等效应的关系进入后续建模判断。

表3中滞后 1 日相关为 0.606–0.759，滞后 7 日仍为 0.476–0.564；六类在这两个滞后下的 95% 区间均不含 0，FDR 校正 $q < 0.001$ 。全部 24 个滞后相关也均满足 $q < 0.05$ 与 $|\rho| \geq 0.30$ 。这既说明销量具有短期惯性和周周期，也意味着普通随机划分会把未来相邻日信息泄漏到训练集，因此问题二必须采用滚动时间验证。

表 3 品类日净销量滞后相关的 HAC 推断

品类	ρ_1 及 95% 区间	FDR q 值	ρ_7 及 95% 区间	FDR q 值
花叶类	0.606 [0.370, 0.841]	< 0.001	0.540 [0.337, 0.743]	< 0.001
花菜类	0.683 [0.601, 0.766]	< 0.001	0.476 [0.355, 0.596]	< 0.001
水生根茎类	0.727 [0.589, 0.864]	< 0.001	0.512 [0.353, 0.672]	< 0.001
茄类	0.704 [0.612, 0.797]	< 0.001	0.548 [0.399, 0.698]	< 0.001
辣椒类	0.759 [0.623, 0.895]	< 0.001	0.540 [0.400, 0.681]	< 0.001
食用菌	0.708 [0.554, 0.862]	< 0.001	0.564 [0.434, 0.694]	< 0.001

5.3 品类间关联及其解释边界

先计算同日品类销量的 Pearson 和 Spearman 相关，再分别回归掉线性趋势、星期和月份，考察残差 Spearman 相关。图6显示，控制日历因素后花叶类—辣椒类、花叶类—食用菌和辣椒类—食用菌仍分别约为 0.68、0.63 和 0.62，可能反映共同客流或联动消费。水生根茎类—茄类原始 Spearman 相关约为 -0.19 ，控制后接近 0.01，原负相关主要由季节错位导致，不能作为稳定替代关系。

对残差秩变量采用同样的 28 阶 HAC 推断，并对 15 组检验控制 FDR。表4列出同时满足 $q < 0.05$ 与 $|\rho| \geq 0.30$ 的 10 组关系；另 5 组虽有部分较小 p 值，但效应量低于 0.30，故不作可执行的共动判断。该双重标准把“统计上可检出”与“经营上有实际强度”区分开来。

表 4 控制日历因素后具有决策意义的品类残差相关

品类组合	Spearman ρ	95% 区间	FDR q 值	效应量
花叶类—辣椒类	0.676	[0.567, 0.785]	< 0.001	强
花叶类—食用菌	0.629	[0.539, 0.718]	< 0.001	强
辣椒类—食用菌	0.621	[0.544, 0.697]	< 0.001	强
花菜类—辣椒类	0.579	[0.478, 0.680]	< 0.001	强
花叶类—花菜类	0.544	[0.430, 0.658]	< 0.001	强
花菜类—食用菌	0.491	[0.409, 0.574]	< 0.001	中等
花叶类—水生根茎类	0.354	[0.245, 0.463]	< 0.001	中等
水生根茎类—食用菌	0.351	[0.238, 0.465]	< 0.001	中等
花菜类—水生根茎类	0.351	[0.258, 0.444]	< 0.001	中等
水生根茎类—辣椒类	0.311	[0.177, 0.445]	< 0.001	中等



图 6 控制日历因素前后的品类销量相关结构

价格、成本和折扣由经营者根据预期需求与品相共同决定，存在明显内生性。图7显示，加成率与销量的观察性 Spearman 相关在不同品类间约为 -0.30 至 0.29 ，控制日历因素后方向和强度还会改变。因此问题一只报告关联；问题二另设双向固定效应模型控制单品和日期异质性，并仍将所得系数称为“条件价格响应”而非因果弹性。

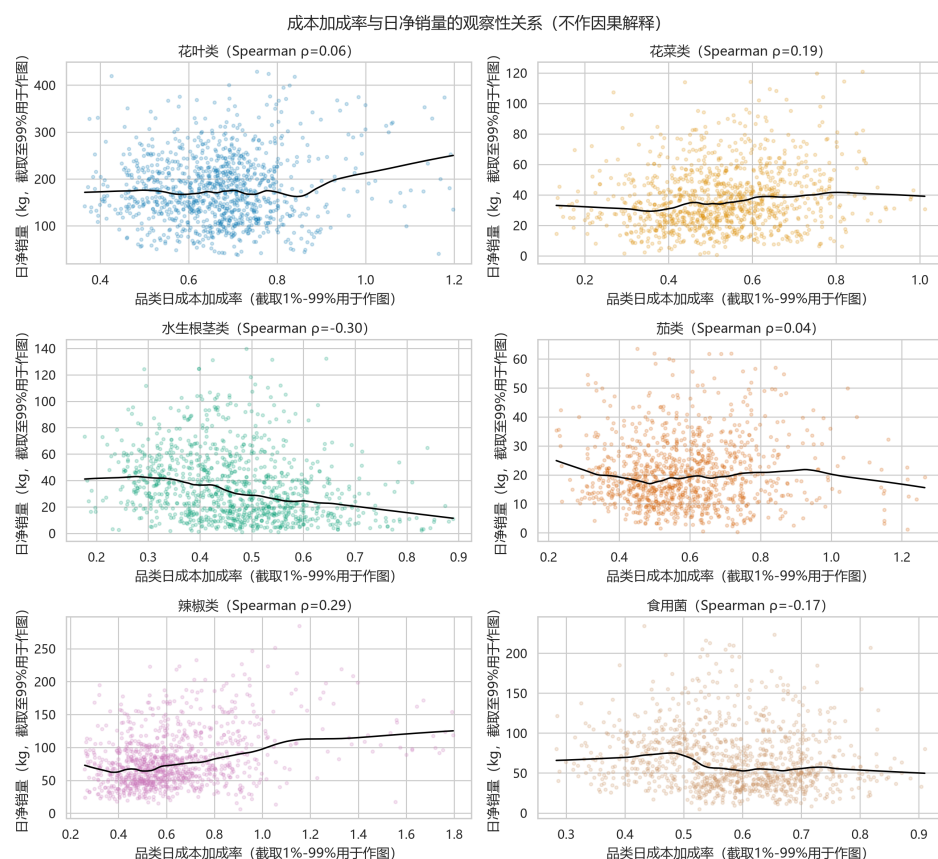


图 7 成本加成率与销量的观察性关系

5.4 单品活跃度与候选集

251 个主数据单品中，246 个出现过销售，5 个无历史销售但仍存在成本和损耗记录。各单品活跃日数差异显著，直接计算 251 维相关矩阵会受到大量结构性零值和短生命周期影响。本文据题意将 2023 年 6 月 24–30 日有正常销售的 49 个单品作为问题三候选集，其中花叶类 17 个、花菜类 2 个、水生根茎类 7 个、茄类 5 个、辣椒类 10 个、食用菌 8 个。该处理将单品层分析从不稳定的两两相关转化为活跃度、近期份额和组合决策。

为量化活跃度对应的补货风险，统计候选单品最近 56 日的正销量天数、正销量变异系数，并以 $2.5/(56\text{日销量}/56)$ 定义“最低陈列覆盖天数”。按活跃日数的三分位点，将不超过 37 日、38–54 日和不少于 55 日分别记为低、中、高活跃。表5显示，低活跃组的正销量 CV 中位数为 0.73，2.5 kg 最低陈列量相当于 4.12 个平均销售日，上四分位数达到 11.53 日；高活跃组相应中位数仅 0.43 和 0.21 日。因此低活跃单品既有更强参数不确定性，也更容易受到最低陈列约束引发的滞销风险，问题三需要用长期份额收缩并进行候选窗口敏感性分析。

表 5 最近 56 日候选单品的活跃度与最低陈列风险

活跃度	单品数	活跃日中位数	正销量 CV 中位数	陈列覆盖天数中位数	上四分位数
低活跃	17	18.0	0.73	4.12	11.53
中活跃	10	52.5	0.56	0.80	1.39
高活跃	22	56.0	0.43	0.21	0.39

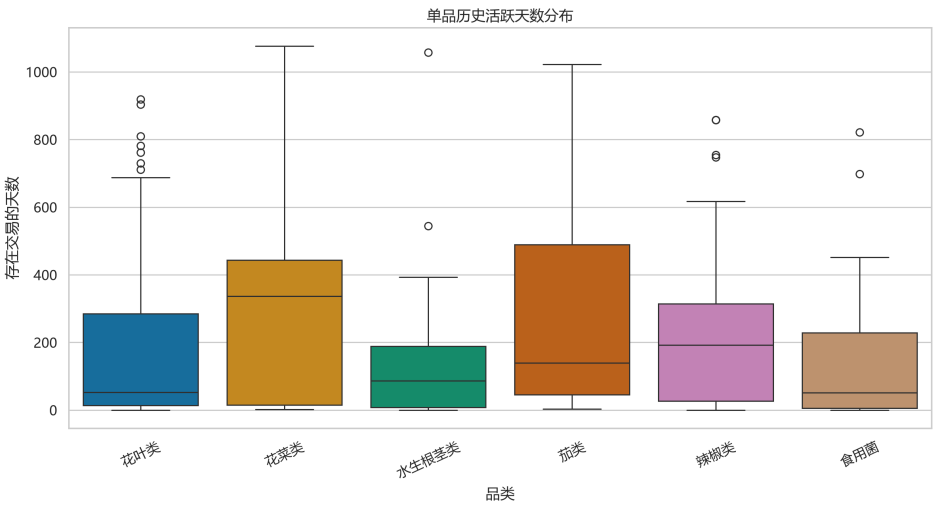


图 8 单品历史活跃度与近期候选集合

图8进一步说明候选单品的历史活跃度并不均衡，近期可售不代表长期稳定。该现

象要求问题三既不能只保留高销量单品，也不能对全部候选单品使用同一份额估计强度。

5.5 问题一对后续建模的约束

上述结果直接决定后续模型结构：右偏长尾支持对数响应和中位数基线；显著的 1 日、7 日动态依赖要求采用时间滚动验证，并使 7 日成为联合残差抽样的自然块长；控制日历后仍存在的中强共动要求需求与成本按同一日期块联合抽取，而不能逐品类独立生成情景；单品活跃度差异则支持问题三采用近期候选集和经交叉验证的份额收缩。价格—销量关系在控制变量变化后并不稳定，因此只进入问题二的条件价格响应情景，不作因果解释。

六、 问题二：品类需求预测及定价补货优化

6.1 预测模型的建立

设销量响应 $z_{c,t} = \log(1 + Y_{c,t})$ ，成本响应 $z_{c,t} = \log W_{c,t}$ 。为显式描述趋势、周内与年周期以及短期惯性，构造动态回归

$$\begin{aligned}
 z_{c,t} = & \beta_{0,c} + \beta_{1,c}t + \sum_{j=1}^6 \gamma_{c,j} \mathbb{I}(\text{weekday}_t = j) \\
 & + \sum_{k=1}^2 \left[a_{c,k} \sin\left(\frac{2\pi k d_t}{365.25}\right) + b_{c,k} \cos\left(\frac{2\pi k d_t}{365.25}\right) \right] \\
 & + \delta_c^\top S_{c,t-7} + \sum_{\ell \in \{1, \dots, 7, 14\}} \phi_{\ell,c} z_{c,t-\ell} + \varepsilon_{c,t}.
 \end{aligned} \tag{3}$$

其中 $S_{c,t-7}$ 为在预测日前一周期已知的成本、加成率和折扣占比状态；另一规格不含状态项。变量并非由逐步回归机械筛选：对数响应来自问题一的右偏长尾；线性趋势、星期虚拟变量与年周期 Fourier 项分别对应长期变化、周内规律和月度季节性；1–7 日及 14 日滞后对应显著的相邻日惯性、完整周依赖和两周记忆。状态项只使用预测时已经可见的滞后信息，以避免把未来价格或成本泄漏进需求预测。更高阶 Fourier 项、超过 14 日的密集滞后及同期价格变量因样本外误差未改善、共线性增加或存在信息泄漏风险而不进入最终模型。参数采用条件最小二乘估计，标准误使用 7 阶 HAC 修正；缺失响应及其受影响的滞后方程直接退出估计，不对原序列插值。模型形式参考动态回归与时间序列回归思想 [7]。

简单基线为最近 8 个相同星期的销量中位数；直方图梯度提升树仅作为非线性备选。四个滚动预测原点依次为 2023 年 6 月 2、9、16 和 23 日，每次仅使用预测原点及

以前数据，递归预测随后 7 日。以

$$\text{WAPE} = \frac{\sum_t |Y_t - \hat{Y}_t|}{\sum_t |Y_t|} \quad (4)$$

为主要选模指标，同时报告 MAE、RMSE 和偏差。提升树虽可拟合非线性，但在仅四个连续滚动原点下稳定性和可解释性均不占优，故不进入最终策略。最终运营模型在动态回归与同周几中位数之间逐品类选择滚动 WAPE 较低者，避免用样本内拟合优度替代真实的未来预测能力。

表6汇总逐品类滚动选模结果及目标周总需求。

表 6 品类需求的滚动验证选模与目标周预测

品类	实际采用模型	滚动 WAPE/%	七日预测总量/kg
花叶类	同周几 8 周中位数	17.16	1118.02
花菜类	动态回归（状态项）	30.44	184.53
水生根茎类	同周几 8 周中位数	27.83	101.59
茄类	同周几 8 周中位数	22.46	170.32
辣椒类	同周几 8 周中位数	15.81	626.43
食用菌	动态回归（状态项）	21.93	359.23

六类需求的逐品类选择后平均 WAPE 为 22.6%，低于统一使用动态回归的 25.1% 和统一使用同周几中位数的 23.9%。花菜类和食用菌采用动态回归，其余品类采用同周几中位数。该结果说明动态结构真实存在，但复杂模型并不必然提高所有品类的样本外精度。

四个七日滚动窗口：实际销量与候选模型预测

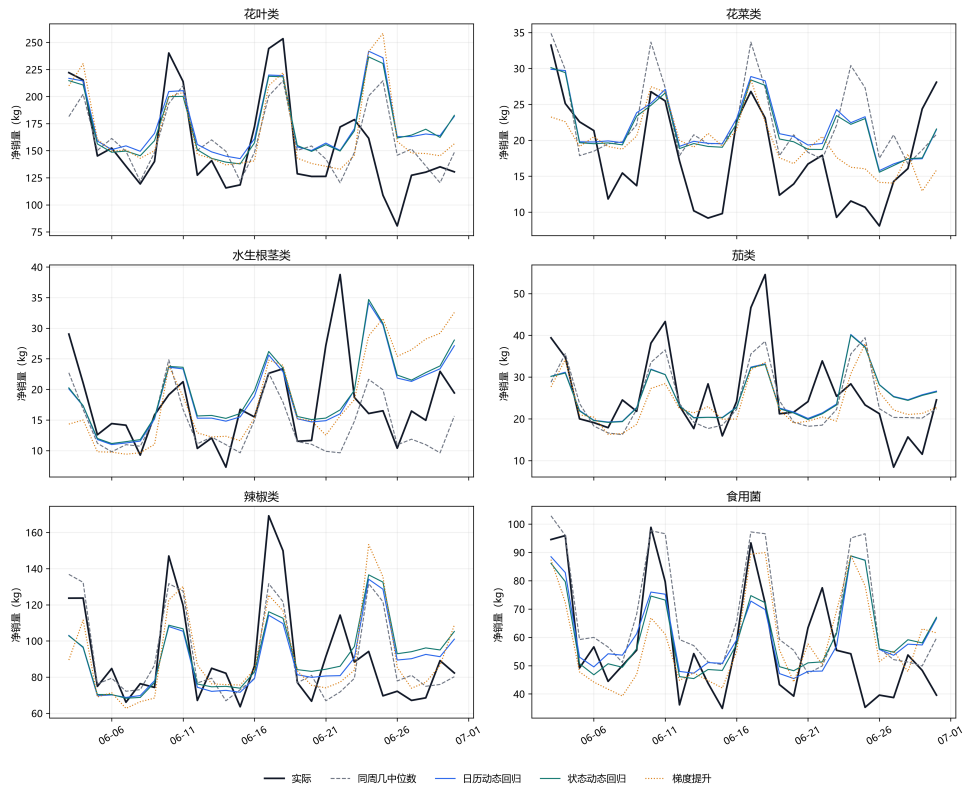


图 9 连续七日滚动验证中的预测与实际销量

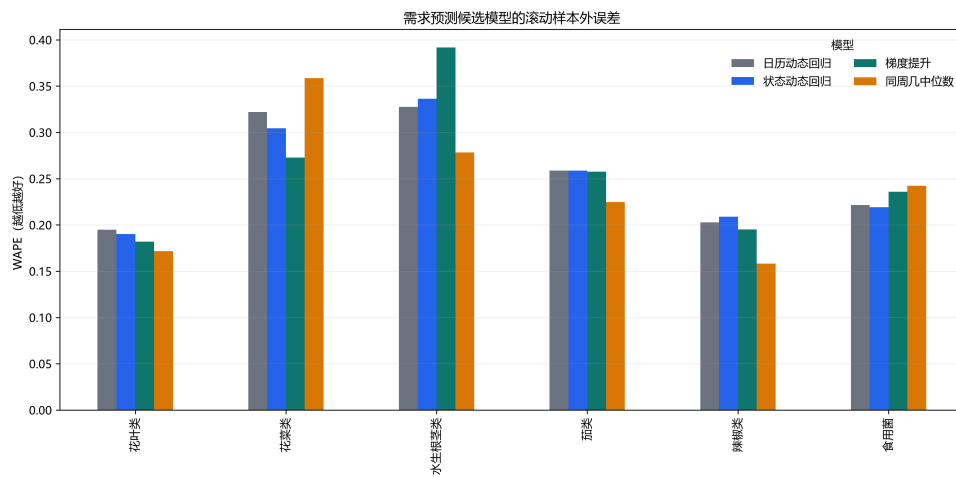


图 10 候选模型的滚动预测误差比较

图9展示各滚动预测周内预测曲线与实际销量的对应关系，可见主要误差集中在少量需求尖峰；图10则比较统一使用各候选模型和逐品类选择后的误差，逐品类选择在保持结构透明的同时取得最低平均 WAPE。

6.2 平稳性、残差与区间诊断

对 12 条“品类 × 目标”序列分别进行 ADF 和 KPSS 检验。水平序列仅 5/12 条经 ADF 拒绝单位根、3/12 条经 KPSS 未拒绝趋势平稳；7 阶季节差分后两项均为 12/12，支持在模型中显式加入趋势、星期和动态滞后。需求模型的 6 个自回归特征根最小模均大于 1，递归预测稳定。

对残差分别进行自相关、正态性、异方差和波动聚集检验，具体采用 Ljung–Box、J–B、B–P 与 ARCH–LM 统计量；同时以 Benjamini–Hochberg 方法控制同类多重检验的假发现率 [6]。FDR 校正后，6/6 个需求模型在 7 阶和 14 阶均未发现显著剩余自相关；但残差仍存在非正态和波动聚集。为此，本文不依赖正态区间，而使用 2000 次移动块残差自助递归预测 [8]。问题一显示六类销量的 7 日滞后相关均显著，且预测期恰为 7 日，因此主模型取 7 日块长：每个抽样块包含一个完整周，既保留周内持续性，也避免逐日独立抽样打散跨品类共同冲击。1 日和 3 日块长另在敏感性分析中比较。图11表明核心动态结构已基本被解释，预测可靠性最终仍以滚动误差和情景评价为准。

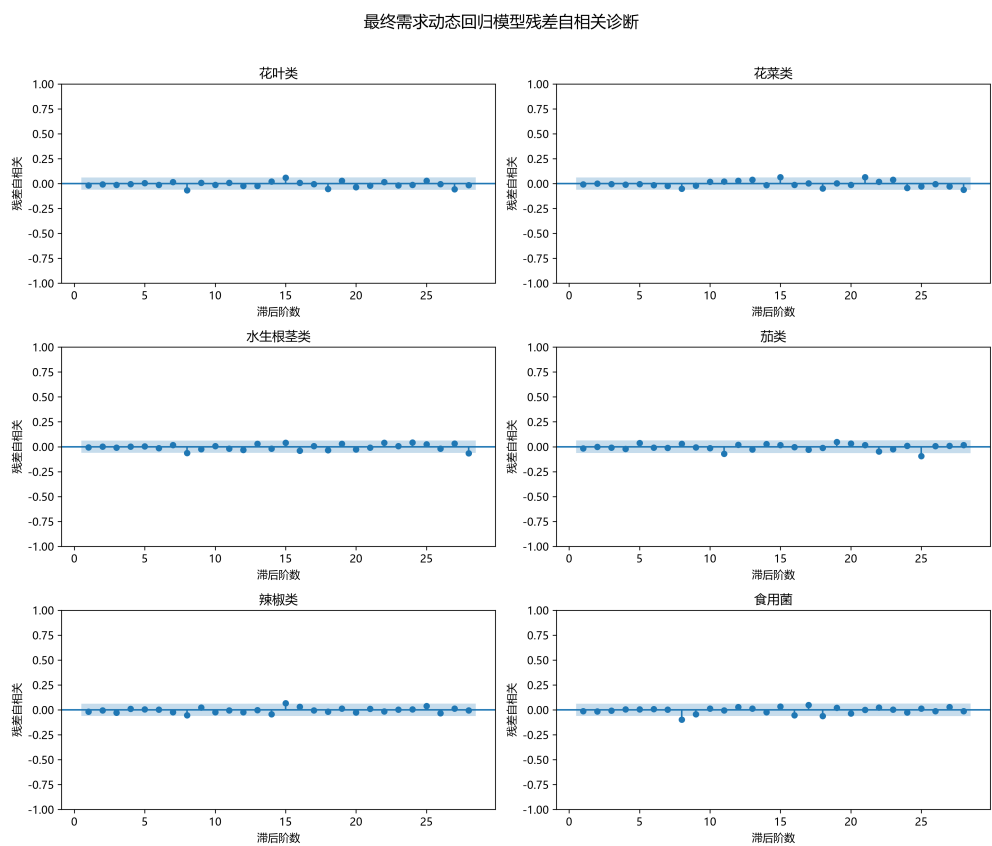


图 11 需求模型残差自相关函数

目标周七日需求点预测分别为：花叶类 1118.02 kg、花菜类 184.53 kg、水生根茎类 101.59 kg、茄类 170.32 kg、辣椒类 626.43 kg、食用菌 359.23 kg。逐日 95% 区间端点之和仅作为风险范围，不解释为七日总量联合置信区间。图12给出需求和成本的逐日走势。

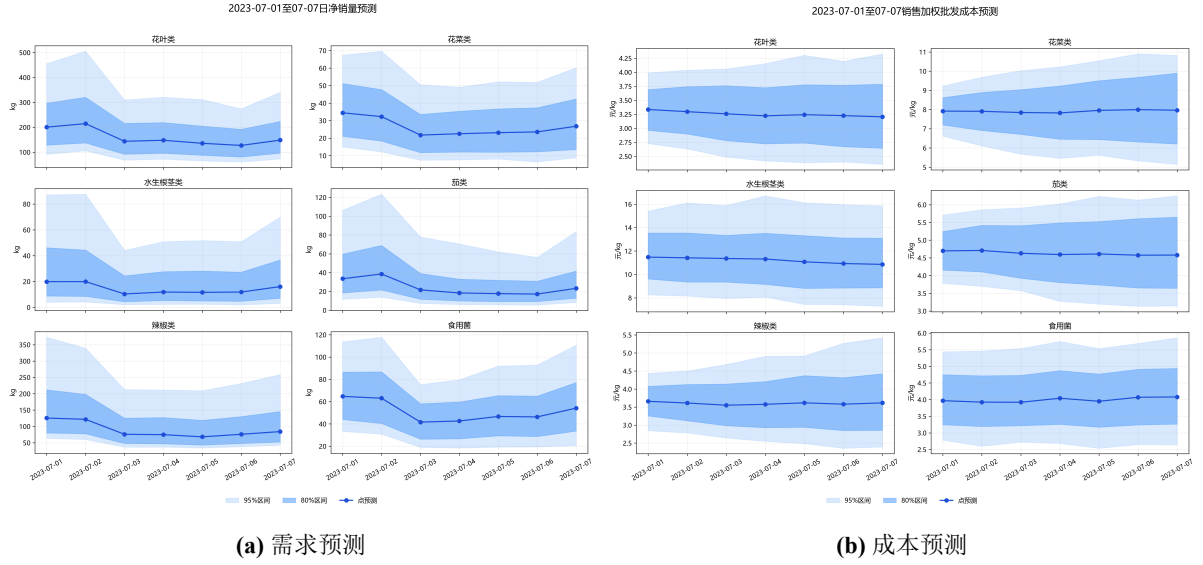


图 12 2023 年 7 月 1-7 日品类需求与成本预测

6.3 条件价格响应估计

为减少跨单品质量差异与共同日期冲击的混杂，在单品日层面建立双向固定效应模型

$$\log(1 + Y_{i,t}) = \beta_{g(i)} \log \frac{P_{i,t}}{W_{i,t}} + \theta_{g(i)} \log W_{i,t} + \eta_{g(i)} R_{i,t} + \mu_i + \tau_t + u_{i,t}, \quad (5)$$

其中 $R_{i,t}$ 为折扣交易占比， μ_i 控制单品长期异质性， τ_t 控制全店共同客流、季节和经营冲击，标准误按单品聚类。分别使用全样本、近 730 日和近 365 日估计，构造弱、中、强三档参数；若窗口方向或显著性不稳定，则弱情景设为零。

表7给出各品类全样本估计及据不同窗口构造的弱、中、强三档响应参数。

表 7 条件价格响应估计及三档参数情景

品类	全样本系数	p 值	弱响应	中响应	强响应
花叶类	-0.747	<0.001	-0.330	-0.661	-0.991
花菜类	-0.155	0.183	0.000	-0.063	-0.126
水生根茎类	-0.843	0.002	-0.393	-0.785	-1.178
茄类	-0.852	<0.001	-0.367	-0.734	-1.101
辣椒类	-0.466	<0.001	-0.233	-0.466	-0.699
食用菌	-0.532	<0.001	-0.200	-0.400	-0.600

除花菜类外，其余五类三窗口均为负且全样本 $p \leq 0.10$ 。花菜类方向不稳定，故不将其负向关系强制外推。即使加入固定效应，仍无法控制未观测的库存、鲜度、陈列和

管理者预期，因此式(5)只用于受限价格情景，不作“提价导致销量下降”的严格因果表述。

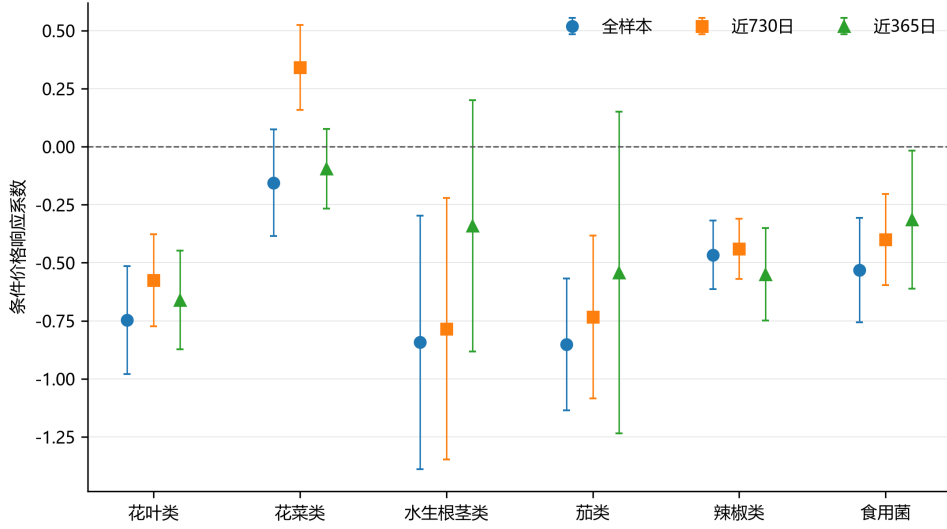


图 13 价格响应系数在三个样本窗口中的稳定性

图13显示花菜类的窗口估计方向与幅度最不稳定，其弱情景因此设为零；其他品类虽总体为负，但区间宽度仍提示价格响应不能被当作固定常数。故优化中抽取三档参数，而不是只代入单一回归点估计。

6.4 联合情景的构造

从滚动验证得到六品类需求与成本的联合对数残差向量

$$\mathbf{e}_t = (e_{1,t}^D, \dots, e_{6,t}^D, e_{1,t}^W, \dots, e_{6,t}^W)^\top. \quad (6)$$

对 28 个连续验证日进行长度为 7 日的移动块抽样，每个抽中块同时保留全部品类和需求—成本分量，从而保留周内持续性及共同冲击。7 日块长同时对应问题一检出的周周期与问题二的完整决策周期；它不是不可检验的固定常数，后文用 1 日、3 日与 7 日块进行敏感性比较。再等概率抽取弱、中、强价格响应参数，共生成 600 个情景。情景 s 中的需求为

$$D_{c,t,s}(m) = D_{c,t,s}^0 \exp \left\{ \beta_{c,s} \log \frac{1+m}{1+m_c^0} \right\}, \quad (7)$$

其中 m_c^0 为历史参考加成率。售价仅由预测成本确定， $P_{c,t} = \widehat{W}_{c,t}(1 + m_{c,t})$ ；实际成本情景 $W_{c,t,s}$ 只进入利润，避免使用预测日尚未实现的成本定价。

6.5 随机优化模型

对每个品类一日期建立历史加成率 10%–90% 加权分位数构成的 7 档价格，以及围绕情景均值需求设置的离散补货量。记选项 k 对应 $(m_{c,t,k}, Q_{c,t,k})$ ，二元变量 $x_{c,t,k}$ 表示

是否选择。预先计算

$$S_{c,t,k,s} = \min\{D_{c,t,s}(m_{c,t,k}), (1 - \ell_c)Q_{c,t,k}\}, \quad (8)$$

$$B_{c,t,k,s} = \min\{S_{c,t,k,s}, D_{c,t,s}^0\}, \quad (9)$$

$$\pi_{c,t,k,s} = P_{c,t,k}S_{c,t,k,s} - W_{c,t,s}Q_{c,t,k}. \quad (10)$$

其中 $B_{c,t,k,s}$ 表示实际满足的基准需求，既不允许通过提价缩小分母，也不会因降价后的新增需求使覆盖率超过 100%。定义

$$R_c(x) = \frac{\sum_{t,k} \bar{B}_{c,t,k} x_{c,t,k}}{\sum_t \bar{D}_{c,t}^0}. \quad (11)$$

对给定覆盖下限 a ，求解

$$\max_x \quad \frac{1}{N} \sum_{s=1}^N \sum_{c,t,k} \pi_{c,t,k,s} x_{c,t,k}, \quad (12)$$

$$\text{s.t.} \quad \sum_k x_{c,t,k} = 1, \quad \forall c, t, \quad (13)$$

$$R_c(x) \geq a, \quad \forall c, \quad (14)$$

$$x_{c,t,k} \in \{0, 1\}. \quad (15)$$

令 a 从 0.70 逐步增加至可行上界，形成最低覆盖率一期望利润 Pareto 前沿。将覆盖率和利润归一化后，以各前沿点到两端连线的垂直距离衡量弯曲程度，距离最大的点作为经营偏好信息缺失时的算法折中点。若门店能够量化缺货成本或服务目标，则可直接据此在前沿上选择相应节点。另用利润 10% 分位数和

$$\text{CVaR}_{10}(\Pi) = \mathbb{E}[\Pi \mid \Pi \leq q_{0.10}(\Pi)] \quad (16)$$

评价下行风险；模型的优化目标仍为情景期望利润。

6.6 效率前沿与主方案

Pareto 前沿的利润端点对应期望利润 6295.72 元、最低品类基准覆盖率 60.43%，且 42 个价格均位于历史支持域上界；覆盖端点附近的最低覆盖率接近 100%，期望利润则降至 -2434.72 元。两个端点表明利润与供给保障之间存在明显冲突。表8进一步比较朴素规则、利润优先、Pareto 折中和最大覆盖附近四种代表方案。

表 8 问题二代表性经营方案比较

方案	期望利润/元	CVaR ₁₀ /元	最低基准覆盖率/%	价格触及上界/%
朴素规则	3026.46	1329.73	74.29	0.00
利润优先	6295.72	4624.11	60.43	100.00
Pareto 折中	4770.83	2630.04	86.00	66.67
最大覆盖附近	-2434.72	-4692.14	100.00	0.00

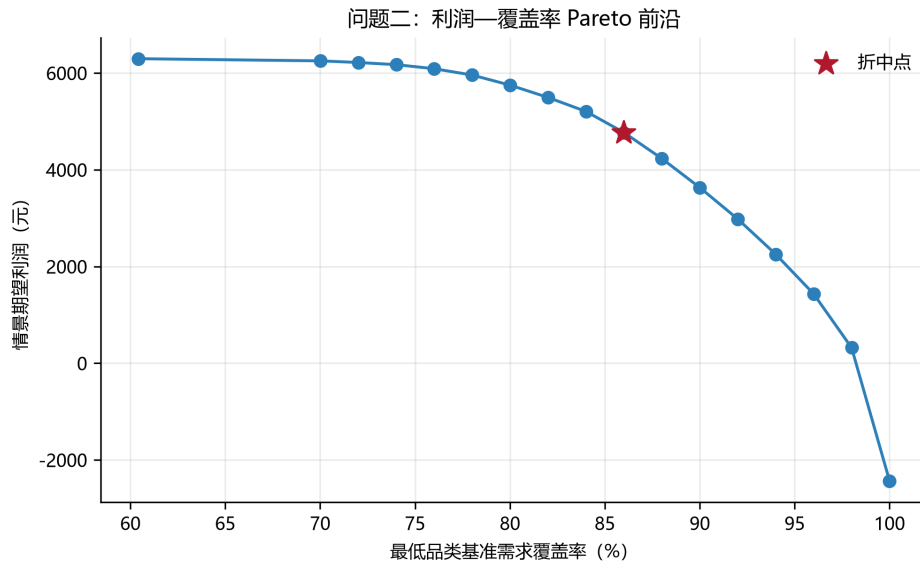


图 14 问题二最低基准覆盖率与期望利润的 Pareto 前沿

图14显示前沿在约 86% 附近开始明显弯折：继续提高最低覆盖率需要越来越多地牺牲利润。该形状而非预设服务阈值构成折中方案的选择依据；若经营者具有明确偏好，可沿同一前沿选择其他节点。

前沿最大弦距位于 86% 覆盖下限。折中方案未来一周总补货 2826.95 kg，情景期望利润 4770.83 元，利润 10% 分位数 3407.21 元，CVaR₁₀ 为 2630.04 元；总体基准覆盖率 86.03%，最低品类覆盖率 86.0005%。相对朴素规则的 3026.46 元，利润提高 57.64%。表9中水生根茎类期望利润为 -306.06 元，说明该品类是维持六类蔬菜覆盖均衡时的主要机会成本来源。

表 9 问题二 Pareto 折中方案的品类汇总

品类	补货量/kg	基准需求/kg	期望销量/kg	期望利润/元	基准覆盖率/%
花叶类	1217.88	1085.43	933.51	1403.59	86.00
花菜类	155.22	142.02	122.24	532.08	86.07
水生根茎类	165.04	129.83	111.67	-306.06	86.01
茄类	204.51	188.43	162.06	411.47	86.01
辣椒类	752.75	646.91	556.35	1962.20	86.00
食用菌	331.54	350.33	301.93	767.55	86.19

水生根茎类出现负利润并不意味着门店应立即取消该品类。现实经营中，基本品类齐全可能承担顾客引流、购物篮完整性和门店形象等未被单品利润记录的价值；但若长期连续亏损，则应通过缩小陈列、调整采购渠道或接受较低覆盖水平进行修正。因此，模型把该负利润解释为供给均衡的显性成本，并将不同覆盖选择完整留在 Pareto 前沿上，而不是把亏损品类自动删除。

七、问题三：单品组合、定价与补货

7.1 候选集与单品需求份额

候选集 $\mathcal{A}$ 为 2023 年 6 月 24–30 日有正常销售的 49 个单品。单品日序列比品类序列稀疏，若对每个 SKU 分别拟合复杂时间序列会产生严重参数不稳定。因此本文先使用问题二 7 月 1 日的品类情景需求，再将其按收缩份额分配到单品。

设 V_i^{56} 为最近 56 日单品净销量， w_i^{365} 为近 365 日在品类内的长期份额， $V_c^{56} = \sum_{i:g(i)=c} V_i^{56}$ 。令 $\kappa_c = \rho_c V_c^{56}$ ，定义

$$\tilde{w}_i = \frac{V_i^{56} + \kappa_c w_i^{365}}{V_c^{56} + \kappa_c}, \quad \sum_{i:g(i)=c} \tilde{w}_i = 1. \quad (17)$$

候选 $\rho_c \in \{0, 0.05, 0.10, 0.20, 0.50, 1.00\}$ 。在预测日前设置 12 个滚动周，以各起点之前 56 日和 365 日数据预测下一周品类内份额，按实际下一周销量加权的交叉熵选择 ρ_c 。这使收缩强度由样本外份额预测而非统一常数决定，而不是人为固定 κ ；其中 $\kappa_c = \rho_c V_c^{56}$ 使先验强度随品类近期规模自动调整。问题四若取得真实缺货与在售状态，可进一步替换为考虑截尾和替代的需求模型 [9]。

7.2 单品 MILP 模型

对每个单品 i 设置未选择选项和若干“价格—补货量”选项 $h \in \mathcal{H}_i$ 。单品加成率取其历史 20%、40%、60%、80% 加权分位数及问题二品类策略，共同限制在历史支持域。补货量包含 2.5 kg 最低陈列量及围绕需求均值的 0.75、1.00、1.25、1.50 倍水平。

令 $y_{i,h} \in \{0, 1\}$ 表示选择选项 h ， z_i 表示单品是否上架，并令 $B_{i,h,s} = \min\{S_{i,h,s}, D_{i,s}^0\}$ 为实际满足的单品基准需求。第一阶段求解

$$\max_{y, \lambda} \quad \lambda, \quad (18)$$

$$\text{s.t.} \quad \sum_{h \in \mathcal{H}_i} y_{i,h} = 1, \quad \forall i, \quad (19)$$

$$z_i = \sum_{h: \text{selected}(h)=1} y_{i,h}, \quad (20)$$

$$27 \leq \sum_i z_i \leq 33, \quad (21)$$

$$Q_i \geq 2.5z_i, \quad (22)$$

$$\sum_{i: g(i)=c} \sum_h \bar{B}_{i,h} y_{i,h} \geq \lambda \sum_{i: g(i)=c} \bar{D}_i^0, \quad \forall c. \quad (23)$$

得到 λ^* 后，第二阶段在式(23)中约束 $\lambda \geq \lambda^* - 10^{-6}$ ，再最大化情景期望利润 $N^{-1} \sum_s \sum_{i,h} \pi_{i,h,s} y_{i,h}$ 。未选择单品的潜在需求仍保留在右端，防止通过删除难卖单品虚增覆盖率。模型在 SciPy 的 MILP 接口中调用 HiGHS 求解，两个阶段均成功，并经回代确认第二阶段保持第一阶段最优覆盖水平。

7.3 问题三结果

第一阶段得到最大最低品类覆盖率 $\lambda^* = 93.6455\%$ 。第二阶段选择 33 个单品，总补货 636.52 kg，期望销量 438.83 kg，期望损耗 141.42 kg，期望利润 427.41 元，利润 10% 分位数为 -153.43 元，CVaR₁₀ 为 -337.43 元；实际最低覆盖率仍为 93.6455%，所有入选单品补货量均不低于 2.5 kg。近期销量前 33 方案利润为 597.59 元但最低覆盖率仅 68.15%，纯利润 MILP 虽达到 1298.41 元，最低覆盖率更降至 45.97%。主方案利润较低是题目所要求的覆盖优先顺序所致。表10给出主方案的分品类 SKU 数、补货量、销量、利润与覆盖率。

表 10 问题三两阶段主方案的品类汇总

品类	SKU 数	补货量/kg	期望销量/kg	期望利润/元	基准覆盖率/%
花叶类	10	257.77	185.75	101.86	93.65
花菜类	2	35.40	26.05	113.56	94.31
水生根茎类	6	49.72	25.06	-255.33	93.65
茄类	3	44.29	32.63	64.16	93.68
辣椒类	7	174.17	108.13	313.63	93.65
食用菌	5	75.16	61.22	89.53	93.66

水生根茎类在单品层面的期望利润为 -255.33 元，是提高最差品类覆盖率的主要代价。六品类有界覆盖率均位于 93.65% 附近，说明第一阶段的最大最小目标确实均衡了品类供给，而不是由高销量品类掩盖低覆盖品类。

八、敏感性分析与模型评价

8.1 预测口径与联合情景边界

预测模型比较全历史、最近 730 日、最近 365 日、毛销量替代净销量以及将极端标记日置为缺失四种口径，目标周总需求相对全历史基准的最大绝对变化为 39.1%。这表明某些品类对训练窗口和尾部样本敏感。本文利用滚动残差和多档价格响应构造联合情景，使上述不确定性直接进入补货与定价优化。

联合情景保留需求与成本残差的周内连续性和跨品类共同冲击，但其信息源仍是 28 个连续验证日，无法覆盖长期供应中断。价格响应也只代表历史条件关联，因此本文不把某一个情景均值解释为确定结果，而同时报告期望利润、10% 分位数和 $CVaR_{10}$ 。折中方案的 $CVaR_{10}$ 为 2630.04 元，说明当前情景范围内尾部平均利润仍为正。

为检验 7 日块长对主方案的影响，在相同 86% 最低覆盖水平下分别采用 1 日、3 日和 7 日块重新生成 600 个联合情景并求解。表 11 显示，三种块长的期望利润位于 4770.83–4910.14 元，最大差异为 2.92%，最低覆盖率均保持 86%；但 $CVaR_{10}$ 在 2300.47–3179.51 元之间变化，说明期望收益较稳定，尾部风险对时间依赖保留方式更敏感。7 日块同时对应显著周周期和完整预测期，因此仍作为主设定。

表 11 86% 覆盖水平下移动块长度敏感性

块长/日	期望利润/元	CVaR ₁₀ /元	最低覆盖率/%	补货量/kg	与 7 日同价比例/%
1	4910.14	3179.51	86.00	2754.53	78.57
3	4804.07	2300.47	86.00	2722.99	80.95
7	4770.83	2630.04	86.00	2826.95	100.00

8.2 覆盖率—利润权衡与尾部风险

表12列出 Pareto 前沿的代表节点。最低覆盖下限由 80% 提高到 90% 时，期望利润由 5749.72 元下降到 3631.77 元；提高到 94% 时利润为 2253.20 元，CVaR₁₀ 仅 138.23 元；达到 98% 时，虽然期望利润仍为 319.72 元，但 CVaR₁₀ 已降至 -1940.02 元。由此可见，在当前需求和成本情景下，追求接近完全覆盖会显著增加下行风险。

表 12 问题二 Pareto 前沿关键节点

覆盖下限/%	期望利润/元	CVaR ₁₀ /元	补货量/kg
80.00	5749.72	3709.69	2551.26
86.00	4770.83	2630.04	2826.95
90.00	3631.77	1530.50	2983.17
94.00	2253.20	138.23	3188.76
98.00	319.72	-1940.02	3562.66

86% 折中点由归一化前沿的最大弦距准则确定，是经营偏好信息缺失条件下的可复现推荐。若商超能够给出缺货成本、最低陈列标准或客户服务目标，则应在前沿上选择与其经营目标相匹配的节点；本文同时报告利润端点、折中点和高覆盖节点，以完整呈现可行决策边界。

8.3 SKU 数量与份额收缩强度

将 SKU 数依次固定为 27–33，并对每个数量分别执行两阶段优化。最大最低覆盖率由 27 个时 86.86% 提高到 33 个时 93.65%，总体随 SKU 数增加而改善；第二阶段利润并不单调，其中 32 个方案覆盖率 92.14%、利润 573.14 元、CVaR₁₀ 为 -140.59 元，而 33 个方案覆盖率更高但利润降至 427.41 元。按题面“尽量满足各品类需求”的优先顺序，33 个是主方案；若经营者允许牺牲约 1.50 个百分点覆盖率，32 个是更有利的经营备选。

表13列出 27–33 个 SKU 的两阶段结果，图15进一步展示覆盖率和利润随 SKU 数量变化的不同方向。

表 13 问题三 SKU 数量前沿

SKU 数	补货量/kg	第二阶段利润/元	CVaR ₁₀ /元	最大最低覆盖率/%
27	577.86	558.49	-73.55	86.86
28	580.82	547.19	-127.24	89.34
29	625.42	365.03	-351.47	90.61
30	582.64	549.52	-140.19	90.69
31	617.55	518.10	-184.78	91.23
32	603.26	573.14	-140.59	92.14
33	636.52	427.41	-337.43	93.65

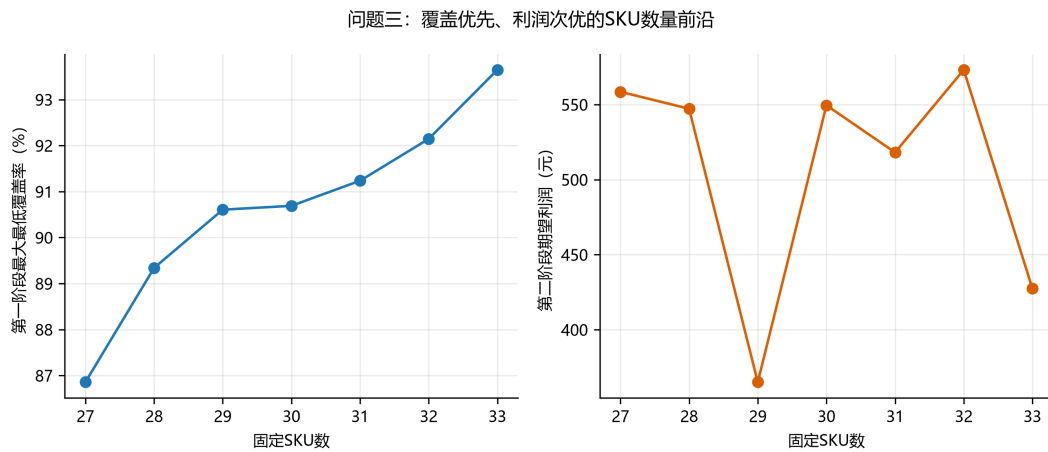


图 15 不同 SKU 数量下的最大最低覆盖率与第二阶段利润

由图15可见，增加 SKU 总体提高最差品类覆盖率，但利润并不单调；这说明更多单品既能改善供给完整性，也会触发最低陈列量和长尾损耗成本。因而 33 个 SKU 是题意见优先顺序下的主方案，32 个 SKU 则是允许少量覆盖损失时的经营备选。

表14给出份额收缩比例 ρ_c 的滚动验证结果：花叶类、水生根茎类和茄类选择 0.20，花菜类和辣椒类选择 0.50，食用菌选择 0.05。各品类最优取值不同，反映了品类内单品结构在短期稳定性和长期代表性上的差异。

表 14 问题三份额收缩强度的滚动交叉验证结果

品类	收缩比例 ρ_c	加权交叉熵	验证起点数
水生根茎类	0.20	1.587	12
花叶类	0.20	2.293	12
花菜类	0.50	0.649	12
茄类	0.20	0.829	12
辣椒类	0.50	2.026	12
食用菌	0.05	2.018	12

为检验收缩参数和候选集窗口带来的解变动，进一步比较 $\rho_c = 0$ 、交叉验证 ρ_c 与 $\rho_c = 1$ ，并将候选窗口由 7 日扩展至 14 日和 28 日。表15显示，不同收缩强度下最低覆盖率均约为 93%，入选重合率不低于 94.12%；候选窗口扩展后入选数分别为 33、31 和 30，最小重合率仍为 94.12%。因此总体覆盖结论较稳健，但边界单品可能因窗口变化替换，实际执行时应在营业前依据最新可供清单滚动重算，而不把一次入选结果长期固定。

表 15 问题三收缩强度与候选窗口敏感性

设定	候选数	最低覆盖率/%	期望利润/元	CVaR ₁₀ /元	入选重合率/%
$\rho_c = 0$	49	93.48	549.27	-193.34	100.00
交叉验证 ρ_c （7 日候选）	49	93.65	427.41	-337.43	100.00
$\rho_c = 1$	49	93.10	353.12	-378.03	94.12
交叉验证 ρ_c （14 日候选）	54	92.14	581.72	-133.20	94.12
交叉验证 ρ_c （28 日候选）	59	91.29	489.63	-224.54	94.12

8.4 模型优点与局限

本文模型有四点优点：第一，所有清洗规则有行数审计，日期缺口不被静默补零；第二，预测模型和份额收缩强度均由滚动样本外表现决定；第三，需求、成本和价格响应在同一场景中联合变化，保留跨品类和周内相关；第四，问题二通过完整效率前沿呈现经营指标之间的权衡，问题三按题意采用覆盖优先的两阶段目标，并对所有整数解进行约束回代。

局限同样明确。其一，观测销量可能被缺货截尾，基准需求仍是历史经营制度下的可售需求近似；其二，双向固定效应不能消除鲜度、陈列和管理者预期导致的价格内生性；其三，品类成本是销售结构加权值，单品预测成本不是供应商正式报价；其四，情

景来自 28 日验证残差，不能覆盖长期供应中断；其五，离散价格与补货网格可能遗漏局部改进；其六，Pareto 折中点只是缺少明确经营偏好时的算法推荐，不能替代管理者对利润、覆盖和风险的最终选择。问题三主方案尾部利润为负，也提示服务优先决策需要与实际风险承受能力共同使用。

九、问题四：建议补充的数据及作用

问题四的核心不是泛泛增加字段，而是让每项数据对应前三问中的具体偏差、参数或约束。按决策价值和采集优先级，建议如下。

9.1 第一优先级：库存、到货和报损流量

- (1) **逐时库存、缺货时点和在售状态。**现有数据只能看到成交量，无法区分零需求、未上架和缺货。库存轨迹可识别销量截尾，估计真实需求，并解释缺货引发的跨品类替代。生鲜品替代和新鲜度差异会系统性影响销量与价格策略解释 [9]。
- (2) **每日订货量、实际到货量、到货时间和拒收量。**这组数据建立“订货一到货一销售一结余”的库存守恒关系，估计供应履约率和随机提前期，并将供应上限、起订量和迟到风险写成硬约束。
- (3) **闭店剩余、报损数量及原因。**目前损耗率是近期盘点的静态比例，无法区分自然腐败、运输破损、盘点误差和主动报废。连续报损数据可建立损耗率随存放时间、温度和鲜度变化的动态模型，并准确估计滞销成本。

9.2 第二优先级：决策时信息集和价格机制

- (1) **凌晨决策时可见的供应商报价、可供量及采购批次。**该数据可替代事后销售结构加权成本，建立采购价格分布和供应商选择模型，避免未来信息泄漏。
- (2) **促销起止时间、折扣幅度、原因和陈列位置。**现有折扣标记不能区分引流促销、临期清仓和质量折价。补充数据后可分离价格、促销、鲜度与陈列的影响，改善条件价格响应的可解释性。
- (3) **单品鲜度、等级、规格、产地和批次。**同名后缀表示不同供应来源，而质量差异同时影响价格、销量和损耗。批次质量可作为控制变量，并支持先进先出和分批降价。

9.3 第三优先级：外部需求和替代关系

天气、节假日、周边活动、门店客流和营业时长可解释共同需求冲击，提升短期预测；匿名购物篮可直接识别互补和替代，而不再依赖品类总销量相关；竞争门店及线上平台价格可建立相对价格响应；冷柜容量、货架面积、配送频次和人工上限可把问题三

的“27-33 个单品”扩展为真实容量约束。生鲜货架分配和替代品定价研究表明，商品组合、陈列空间与价格需要联合考虑 [3, 9]。

若上述数据逐步完备，可将本文单周期框架扩展为部分可观测的多周期库存一定价模型：状态包括库存年龄、鲜度和供应状态，决策包括订货、价格、折扣和报废，目标同时考虑期望利润、缺货成本与损耗成本。实施顺序应先补齐库存守恒和决策时供应信息，再增加促销、客流和竞争价格；否则在基础库存状态不可见时盲目增加外部变量，模型解释仍会受销量截尾限制。

9.4 采集难度与模型调整路径

上述数据并非都能在短期内取得。表16从现实难度、主要收益和暂时缺失时的替代处理三个方面给出实施路径。

表 16 新增数据的现实难度及替代建模策略

数据	难度	主要困难与建模收益	暂时缺失时的处理
逐时库存与缺货	中	需打通 POS 与库存系统并校正盘点误差；可识别销量截尾和真实在售时段	将销量解释为历史制度下的条件需求，扩大需求情景并标记无流水日
订货、到货与供应报价	中	需整合采购系统和供应商回执；可识别提前期、履约率和决策时成本	使用历史成本预测区间，并在供应情景和历史支持域内制定策略
闭店剩余与损耗原因	中高	需坚持逐日盘点并统一报损分类；可建立库存年龄和动态损耗模型	沿用近期损耗率并开展压力测试，不区分自然腐败和主动报废
促销与陈列记录	低中	需规范促销原因、起止时间和陈列位置；可分离价格、促销与鲜度影响	保留折扣标记并采用单品、日期固定效应，避免把相关解释为因果
购物篮与客流	高	涉及交易关联、隐私脱敏和跨系统匹配；可直接识别互补替代及转化率	使用控制日历后的残差相关构造联合情景，不估计交叉价格弹性
天气、节假日与活动	低	公共数据易取得，但需按门店日期对齐并防止事后信息进入训练	使用星期、月份、Fourier 季节项和滚动验证吸收共同冲击

若库存、到货和报损等关键数据能够取得，模型可升级为以库存年龄和鲜度为状态的多周期随机控制模型，并显式计入缺货、持有和报损成本；若高难度的购物篮或实时库存暂时无法获取，则维持本文的短期滚动框架，收缩价格搜索范围、扩大需求情景并报告尾部风险，而不虚构不可识别的替代率或长期需求。这样既给出完善数据条件下的升级方向，也保留现实数据不足时可以执行的简化方案。

十、 结论

本文从 878503 条交易流水出发，在不删除真实极端值、不把缺口补零的前提下，建立品类日和单品日两层分析。问题一表明六类蔬菜需求具有右偏长尾、显著的相邻日惯性和周周期；HAC 推断与 FDR 校正后，15 组残差相关中有 10 组同时达到 $q < 0.05$ 和中等以上效应，支持时间滚动验证和联合随机残差情景。低活跃候选单品的最低陈列覆盖天数明显更长，为问题三的份额收缩和窗口检验提供了风险依据。

问题二通过逐品类滚动选模得到未来一周需求与成本预测，再以双向固定效应构造价格响应情景。在有界基准覆盖率定义下求得利润—覆盖率 Pareto 前沿，最大弦距准则对应最低覆盖率 86.00% 的折中方案。该方案补货 2826.95 kg，期望利润 4770.83 元，较朴素规则提高 57.64%， $CVaR_{10}$ 为 2630.04 元。不同残差块长下期望利润最大差异为 2.92%，但尾部风险对块长更敏感。

问题三以 49 个近期可售单品为候选，通过滚动验证确定品类收缩强度，并以两阶段 MILP 先最大化最低品类覆盖率、再最大化利润。33 个单品主方案的最低覆盖率为 93.65%，期望利润 427.41 元；32 个备选方案覆盖率 92.14%、利润 573.14 元，揭示了满足题目优先顺序所付出的利润和尾部风险代价。所有题面硬约束均经回代验证。

本文结论不是对“真实需求”或“因果价格弹性”的最终识别，而是在现有数据和历史经营制度下可复核的 7 日滚动策略。问题四提出的库存、供应、报损和促销数据将决定模型能否进一步从条件预测走向因果识别与多周期控制；在关键数据不足时，应继续使用受限价格域、扩大随机情景并报告尾部风险，而不将短期结果直接解释为长期规律。

AI 工具使用说明

本项目使用生成式 AI 辅助梳理题目、检查代码、解释统计诊断并润色论文表达。数据口径、模型公式、运行结果、约束回代和最终结论均由本地脚本复现并人工核验；AI 未用于伪造数据、文献或未运行的实验结果。具体披露方式应以实际提交年份的竞赛规则为准。

参考文献

- [1] 唐跃武, 范体军, 程永伟. 基于不同保鲜方式的生鲜品定价订货决策. 中国管理科学, 30(3):189–198, 2022.
- [2] 陈军, 康莎. 面向双渠道销售的农产品定价与库存补货联合决策. 工业工程, 26(3):39–46, 2023.
- [3] 杨帅, 黄湘萌, 王俊斌. 生鲜食品货架分配与定价联合优化策略研究. 供应链管理, 3(8):49–59, 2022.
- [4] 毕文杰, 周玉冰. 基于深度强化学习的生鲜产品联合库存控制与动态定价研究. 计算机应用研究, 39(9):2660–2664, 2022.

- [5] Whitney K. Newey and Kenneth D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. Econometrica, 55(3):703–708, 1987.
- [6] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. Journal of the Royal Statistical Society: Series B, 57(1):289–300, 1995.
- [7] 周永道, 王会琦, 吕王勇. 时间序列分析及应用. 高等教育出版社, 北京, 2015.
- [8] Bradley Efron and Robert J. Tibshirani. An Introduction to the Bootstrap. Chapman and Hall/CRC, 1993.
- [9] 徐兵, 李慧芳. 新鲜度差异下两种可替代生鲜品的多阶段定价策略研究. 工业工程, 24(4):10–19, 2021.
- [10] DeepSeek. DeepSeek-R1-0528. 深度求索 (DeepSeek). 2026-08-29.
- [11] ChatGPT. GPT-5.6. OpenAI. 2026-08-29.

附录 A 问题二每日定价与补货明细

问题二折中方案在未来七日的逐品类补货量、成本加成率和售价见表17，该表可直接作为每日滚动执行的基准清单。

表 17 问题二未来一周每日定价与补货方案

日期	品类	补货/kg	加成率/%	售价/(元/kg)	期望销量/kg	基准覆盖率/%
07-01	水生根茎类	33.27	41.35	16.23	22.48	86.93
07-01	花叶类	251.33	54.46	5.15	193.78	95.83
07-01	花菜类	31.38	81.60	14.38	24.81	88.55
07-01	茄类	39.54	97.80	9.29	31.47	81.70
07-01	辣椒类	147.50	132.61	8.51	113.01	85.70
07-01	食用菌	64.24	71.16	6.79	59.01	88.43
07-02	水生根茎类	32.18	41.35	16.14	21.95	85.03
07-02	花叶类	221.09	101.78	6.65	170.02	79.84
07-02	花菜类	28.51	81.60	14.36	22.23	88.41
07-02	茄类	45.59	97.80	9.32	35.54	80.92
07-02	辣椒类	150.64	132.61	8.40	110.59	86.37
07-02	食用菌	62.59	71.16	6.72	56.73	88.22
07-03	水生根茎类	15.94	41.35	16.06	11.11	86.12
07-03	花叶类	144.09	101.78	6.57	110.53	79.37
07-03	花菜类	16.80	81.60	14.25	13.40	82.09
07-03	茄类	27.03	60.93	7.45	22.27	91.41

续表 17

日期	品类	补货/kg	加成率/%	售价/(元/kg)	期望销量/kg	基准覆盖率/%
07-03	辣椒类	89.69	132.61	8.26	67.23	85.68
07-03	食用菌	37.06	71.16	6.71	34.30	85.10
07-04	水生根茎类	19.52	41.35	16.00	12.89	87.73
07-04	花叶类	164.71	54.46	4.98	130.37	92.40
07-04	花菜类	16.15	81.60	14.21	13.29	79.75
07-04	茄类	22.41	60.93	7.39	18.27	91.88
07-04	辣椒类	88.34	132.61	8.32	65.28	85.94
07-04	食用菌	37.97	71.16	6.92	34.49	84.26
07-05	水生根茎类	18.45	41.35	15.66	12.37	86.89
07-05	花叶类	139.73	101.78	6.54	102.14	79.74
07-05	花菜类	17.86	81.60	14.45	14.29	83.99
07-05	茄类	21.62	60.93	7.42	17.47	90.75
07-05	辣椒类	82.81	132.61	8.41	59.35	86.11
07-05	食用菌	41.59	71.16	6.76	37.66	85.19
07-06	水生根茎类	19.53	41.35	15.45	12.97	85.09
07-06	花叶类	127.50	101.78	6.51	95.50	79.15
07-06	花菜类	20.15	81.60	14.52	15.61	87.44
07-06	茄类	19.96	97.80	9.05	14.89	80.73
07-06	辣椒类	93.81	132.61	8.33	67.24	86.38
07-06	食用菌	41.23	71.16	6.97	37.32	85.57
07-07	水生根茎类	26.14	41.35	15.35	17.91	84.89
07-07	花叶类	169.44	54.46	4.95	131.18	92.92
07-07	花菜类	24.39	81.60	14.46	18.60	88.60
07-07	茄类	28.37	60.93	7.37	22.15	92.12
07-07	辣椒类	99.96	132.61	8.41	73.65	85.82
07-07	食用菌	46.86	71.16	6.98	42.44	84.45

附录 B 问题三单品定价与补货明细

问题三主方案的 33 个入选单品及 7 月 1 日补货量、加成率、售价和期望利润见表18。

表 18 问题三 2023 年 7 月 1 日单品定价与补货方案

单品编码	单品名称	品类	补货/kg	加成率/%	售价/(元/kg)	期望利润/元
102900005116899	净藕(1)	水生根茎类	20.54	34.65	13.28	-63.18
102900051000944	洪湖藕带	水生根茎类	12.76	33.33	22.80	-102.98
102900005118824	高瓜(1)	水生根茎类	8.46	24.95	13.85	-51.77
102900011007969	红莲藕带	水生根茎类	2.96	37.14	6.89	-5.82
102900011001691	菱角	水生根茎类	2.50	41.35	12.30	-9.95
102900011032732	高瓜(2)	水生根茎类	2.50	41.35	18.38	-21.63
102900011030059	云南生菜(份)	花叶类	68.04	54.46	5.75	28.56
102900011030097	云南油菜菜(份)	花叶类	41.17	54.46	4.68	14.06
102900011030110	菠菜(份)	花叶类	32.23	54.46	6.70	15.78
102900005118831	娃娃菜	花叶类	26.58	54.46	7.25	24.76
102900005115786	竹叶菜	花叶类	23.90	54.46	3.54	3.35
102900011008164	奶白菜	花叶类	18.28	54.46	4.22	1.78

续表 18

单品编码	单品名称	品类	补货/kg	加成率/%	售价/(元/kg)	期望利润/元
102900005115762	莴菜	花叶类	15.22	54.46	3.64	0.02
102900011023464	小青菜 (1)	花叶类	14.13	54.46	4.45	4.14
102900005119975	红薯尖	花叶类	9.21	54.46	4.28	3.20
102900005115823	上海青	花叶类	9.01	45.99	6.37	6.20
102900005116714	西兰花	花菜类	25.28	81.60	13.80	53.43
102900011034026	枝江青梗散花	花菜类	10.13	115.37	18.09	60.13
102900005116257	紫茄子 (2)	茄类	35.62	76.24	6.05	38.26
102900011022764	长线茄	茄类	6.18	104.22	14.30	21.78
102900005116509	青茄子 (1)	茄类	2.50	97.80	7.87	4.12
102900011031100	小米椒 (份)	辣椒类	46.27	132.61	4.85	40.80
102900011016701	芜湖青椒 (1)	辣椒类	42.21	132.61	8.34	72.78
102900011032251	螺丝椒 (份)	辣椒类	31.81	132.61	9.86	57.02
102900011032022	小皱皮 (份)	辣椒类	22.86	132.61	4.83	20.06
102900011032848	姜蒜小米椒组合装 (小份)	辣椒类	13.59	132.61	5.35	13.24
102900011000328	螺丝椒	辣椒类	13.33	132.61	20.61	79.83
102900011035078	红椒 (2)	辣椒类	4.10	117.29	27.30	29.90
106949711300259	金针菇 (盒)	食用菌	34.80	71.16	2.18	16.88
106971533450003	海鲜菇 (包)	食用菌	16.71	71.16	2.94	11.08
102900011034330	双孢菇 (盒)	食用菌	13.97	71.16	5.14	24.98
102900005115250	西峡花菇 (1)	食用菌	5.83	71.16	23.49	34.32
102900011031926	虫草花 (份)	食用菌	3.85	71.16	3.92	2.28

附录 C 核心程序代码与复现说明

本附录直接列出产生本文主要统计量和策略结果的核心 Python 代码。为避免将读取配置、绘图和表格排版等重复性辅助程序全部收入论文，代码清单保留数据清洗与连接、动态回归、滚动验证、价格响应、随机情景、Pareto 优化、份额收缩、两阶段 MILP 及稳健性检验的关键实现；完整可执行文件仍位于 code/ 目录。代码中的随机过程均使用固定种子，表格由 code/build_paper_tables.py 根据最终 CSV 自动生成。

C.1 数据清洗、字段转换与同日成本连接

代码 1 完成原始附件读取、字段规范、销售与退货符号处理，以及“销售流水—单品主表—同日批发成本—损耗率”的连接和审计。

代码 1: 数据清洗与同日成本连接核心代码

```

1 def main() -> None:
2     CLEAN_DIR.mkdir(parents=True, exist_ok=True)
3     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
4
5     input_hashes_before = {name: sha256(path) for name, path in INPUTS.items()}
6
7     master_raw = pd.read_csv(INPUTS["附件1.csv"], encoding="utf-8-sig", dtype="string")
8     sales_raw = pd.read_csv(INPUTS["附件2.csv"], encoding="utf-8-sig", dtype="string")
9     cost_raw = pd.read_csv(INPUTS["附件3.csv"], encoding="utf-8-sig", dtype="string")
10    loss_raw = pd.read_csv(INPUTS["附件4.csv"], encoding="utf-8-sig", dtype="string")
11
12    raw_frames = {
13        "附件1.csv": strip_strings(master_raw),
14        "附件2.csv": strip_strings(sales_raw),

```

```

15     "附件3.csv": strip_strings(cost_raw),
16     "附件4.csv": strip_strings(loss_raw),
17 }
18
19 master = add_source_row_number(raw_frames["附件1.csv"]).rename(
20     columns={
21         "单品编码": "item_code",
22         "单品名称": "item_name",
23         "分类编码": "category_code",
24         "分类名称": "category_name",
25     }
26 )
27 sales = add_source_row_number(raw_frames["附件2.csv"]).rename(
28     columns={
29         "销售日期": "sale_date_raw",
30         "扫码销售时间": "sale_time_raw",
31         "单品编码": "item_code",
32         "销量(千克)": "quantity_raw",
33         "销售单价(元/千克)": "sale_unit_price_raw",
34         "销售类型": "transaction_type",
35         "是否打折销售": "discount_flag",
36     }
37 )
38 cost = add_source_row_number(raw_frames["附件3.csv"]).rename(
39     columns={
40         "日期": "wholesale_date_raw",
41         "单品编码": "item_code",
42         "批发价格(元/千克)": "wholesale_unit_cost_raw",
43     }
44 )
45 loss = add_source_row_number(raw_frames["附件4.csv"]).rename(
46     columns={
47         "单品编码": "item_code",
48         "单品名称": "item_name",
49         "损耗率(%)": "loss_rate_pct_raw",
50     }
51 )
52
53 sales["sale_date"] = pd.to_datetime(sales["sale_date_raw"], errors="coerce")
54 sales["quantity_kg"] = pd.to_numeric(sales["quantity_raw"], errors="coerce")
55 sales["sale_unit_price"] = pd.to_numeric(sales["sale_unit_price_raw"], errors="coerce")
56 time_pattern = r"(?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:\.\d{1,9})?"
57 sales["valid_time_format"] = sales["sale_time_raw"].str.fullmatch(time_pattern, na=False)
58 sales["sale_time_precision"] = np.where(
59     sales["sale_time_raw"].str.contains(".", regex=False, na=False),
60     "fractional_seconds",
61     "seconds",
62 )
63 sales["sale_datetime"] = pd.to_datetime(
64     sales["sale_date_raw"].astype("string") + " " + sales["sale_time_raw"].astype("string"),
65     errors="coerce",
66 )
67
68 cost["wholesale_date"] = pd.to_datetime(cost["wholesale_date_raw"], errors="coerce")
69 cost["wholesale_unit_cost"] = pd.to_numeric(
70     cost["wholesale_unit_cost_raw"], errors="coerce"
71 )
72 loss["loss_rate_pct"] = pd.to_numeric(loss["loss_rate_pct_raw"], errors="coerce")
73
74 master_codes = set(master["item_code"])
75 sales_codes = set(sales["item_code"])
76 no_history_codes = sorted(master_codes - sales_codes)
77 master["has_historical_sales"] = master["item_code"].isin(sales_codes)
78

```

```

79 sales_key = ["sale_date", "sale_time_raw", "item_code"]
80 sales["candidate_time_key_group_size"] = sales.groupby(sales_key, dropna=False)[
81     "source_row_number"
82 ].transform("size")
83 sales["candidate_time_key_duplicate_flag"] = sales["candidate_time_key_group_size"].gt(1)
84
85 cost_key = ["wholesale_date", "item_code"]
86 cost["cost_key_duplicate_flag"] = cost.duplicated(cost_key, keep=False)
87
88 loss["zero_loss_flag"] = loss["loss_rate_pct"].eq(0)
89 loss["high_loss_flag"] = loss["loss_rate_pct"].ge(25)
90
91 quantity_threshold = sales.loc[sales["transaction_type"].eq("销售"), "quantity_kg"].quantile(
92     EXTREME_QUANTILE
93 )
94 price_threshold = sales.loc[sales["transaction_type"].eq("销售"), "sale_unit_price"].quantile(
95     EXTREME_QUANTILE
96 )
97 cost_threshold = cost["wholesale_unit_cost"].quantile(EXTREME_QUANTILE)
98 cost["high_cost_flag"] = cost["wholesale_unit_cost"].gt(cost_threshold)
99
100 sales_joined = (
101     sales.merge(
102         master[["item_code", "item_name", "category_code", "category_name"]],
103         on="item_code",
104         how="left",
105         validate="many_to_one",
106     )
107     .merge(
108         cost[["wholesale_date", "item_code", "wholesale_unit_cost"]],
109         left_on=["sale_date", "item_code"],
110         right_on=["wholesale_date", "item_code"],
111         how="left",
112         validate="many_to_one",
113     )
114     .merge(loss[["item_code", "loss_rate_pct"]], on="item_code", how="left", validate="many_to_one")
115 )
116
117 is_sale = sales_joined["transaction_type"].eq("销售")
118 is_return = sales_joined["transaction_type"].eq("退货")
119 is_discount = sales_joined["discount_flag"].eq("是")
120 sales_joined["is_sale"] = is_sale
121 sales_joined["is_return"] = is_return
122 sales_joined["is_discount"] = is_discount
123 sales_joined["discount_sale_indicator"] = is_sale & is_discount
124 sales_joined["cost_match_exact_same_day"] = sales_joined["wholesale_unit_cost"].notna()
125 sales_joined["gross_sales_quantity_kg"] = np.where(is_sale, sales_joined["quantity_kg"], 0.0)
126 sales_joined["return_quantity_abs_kg"] = np.where(is_return, -sales_joined["quantity_kg"], 0.0)
127 sales_joined["net_sales_quantity_kg"] = sales_joined["quantity_kg"]
128 sales_joined["gross_sales_revenue_yuan"] = np.where(
129     is_sale, sales_joined["quantity_kg"] * sales_joined["sale_unit_price"], 0.0
130 )
131 sales_joined["return_refund_abs_yuan"] = np.where(
132     is_return, -sales_joined["quantity_kg"] * sales_joined["sale_unit_price"], 0.0
133 )
134 sales_joined["net_sales_revenue_yuan"] = (
135     sales_joined["gross_sales_revenue_yuan"] -
136     sales_joined["return_refund_abs_yuan"]
137 )
138 sales_joined["sales_cost_weighted_yuan"] = np.where(
139     is_sale, sales_joined["quantity_kg"] * sales_joined["wholesale_unit_cost"], 0.0
140 )
141 sales_joined["gross_margin_yuan"] = (
142     sales_joined["net_sales_revenue_yuan"] -
143     sales_joined["sales_cost_weighted_yuan"]
144 )

```

```

143 sales_joined["price_cost_diff_yuan_per_kg"] = (
144     sales_joined["sale_unit_price"] - sales_joined["wholesale_unit_cost"]
145 )
146 sales_joined["transaction_markup_rate"] = np.where(

```

C.2 动态回归与滚动选模

代码 2 构造趋势、星期、Fourier 项和滞后变量，并实现动态回归递归预测；代码 3 按照连续滚动原点比较动态回归与同周几中位数基线。

代码 2: 动态回归特征构造与多步预测

```

1 def deterministic_features(dates: pd.DatetimeIndex, start_date: pd.Timestamp) -> pd.DataFrame:
2     dates = pd.DatetimeIndex(dates)
3     elapsed_days = (dates - start_date).days.to_numpy(dtype=float)
4     out = pd.DataFrame(index=dates)
5     out["trend_year"] = elapsed_days / 365.25
6     weekday = dates.dayofweek
7     for day in range(1, 7):
8         out[f"weekday_{day}"] = (weekday == day).astype(float)
9     annual_phase = 2.0 * np.pi * dates.dayofyear.to_numpy(dtype=float) / 365.25
10    for harmonic in (1, 2):
11        out[f"annual_sin_{harmonic}"] = np.sin(harmonic * annual_phase)
12        out[f"annual_cos_{harmonic}"] = np.cos(harmonic * annual_phase)
13    return out
14
15
16 def fit_scalings(train: pd.DataFrame, features: list[str]) -> list[Scaling]:
17     scalings: list[Scaling] = []
18     for feature in features:
19         values = pd.to_numeric(train[feature], errors="coerce")
20         lower = float(values.quantile(0.01))
21         upper = float(values.quantile(0.99))
22         clipped = values.clip(lower, upper)
23         mean = float(clipped.mean())
24         std = float(clipped.std(ddof=0))
25         if not np.isfinite(std) or std < 1e-10:
26             std = 1.0
27         scalings.append(Scaling(feature, mean, std, lower, upper))
28     return scalings
29
30
31 def apply_scalings(frame: pd.DataFrame, scalings: list[Scaling]) -> pd.DataFrame:
32     out = frame.copy()
33     for scaling in scalings:
34         values = pd.to_numeric(out[scaling.feature], errors="coerce")
35         values = values.fillna(scaling.mean).clip(scaling.lower, scaling.upper)
36         out[scaling.feature] = (values - scaling.mean) / scaling.std
37     return out
38
39
40 def build_exog(
41     full: pd.DataFrame,
42     dates: pd.DatetimeIndex,
43     start_date: pd.Timestamp,
44     specification: str,
45     scalings: list[Scaling] | None = None,
46 ) -> tuple[pd.DataFrame, list[Scaling]]:
47     exog = deterministic_features(dates, start_date)
48     state_features: list[str] = []

```

```

49     if specification == "state7":
50         lookup = full.set_index("sale_date")
51         lag_dates = pd.DatetimeIndex(dates) - pd.Timedelta(days=7)
52         state = lookup.reindex(lag_dates)[
53             ["weighted_wholesale_cost", "markup_rate", "discount_sale_share"]
54         ].copy()
55         state.index = dates
56         state["lag7_log_wholesale_cost"] = np.log(
57             pd.to_numeric(state["weighted_wholesale_cost"], errors="coerce").clip(lower=1e-8)
58         )
59         state = state.rename(
60             columns={
61                 "markup_rate": "lag7_markup_rate",
62                 "discount_sale_share": "lag7_discount_share",
63             }
64         )
65         state_features = [
66             "lag7_log_wholesale_cost",
67             "lag7_markup_rate",
68             "lag7_discount_share",
69         ]
70         exog = exog.join(state[state_features])
71         if scalings is None:
72             scalings = fit_scalings(exog, state_features)
73         exog = apply_scalings(exog, scalings)
74         return exog.astype(float), scalings or []
75
76
77 def dynamic_regression_forecast(
78     full: pd.DataFrame,
79     target: str,
80     train_end: pd.Timestamp,
81     forecast_dates: pd.DatetimeIndex,
82     specification: str,
83     bootstrap_repetitions: int = 2000,
84     training_window_days: int | None = DEFAULT_TRAINING_WINDOW_DAYS,
85 ) -> tuple[pd.DataFrame, object, list[Scaling], str]:
86     train_mask = full["sale_date"].le(train_end)
87     if training_window_days is not None:
88         train_start = train_end - pd.Timedelta(days=training_window_days - 1)
89         train_mask &= full["sale_date"].ge(train_start)
90     train = full.loc[train_mask].copy()
91     start_date = pd.Timestamp(train["sale_date"].min())
92     train_dates = pd.DatetimeIndex(train["sale_date"])
93     exog_train, scalings = build_exog(full, train_dates, start_date, specification)
94     exog_future, _ = build_exog(full, forecast_dates, start_date, specification, scalings)
95     transformed = pd.Series(transform_target(train[target], target), index=train_dates, name=target)
96     transformed[pd.isna(train[target]).to_numpy()] = np.nan
97     design = exog_train.copy()
98     for lag in DYNAMIC_LAGS:
99         design[f"target_lag_{lag}"] = transformed.shift(lag)
100     design = sm.add_constant(design, has_constant="add")
101     valid = transformed.notna() & design.notna().all(axis=1)
102     result = sm.OLS(transformed.loc[valid], design.loc[valid]).fit(
103         cov_type="HAC", cov_kwds={"maxlags": 7, "use_correction": True}
104     )
105     residuals = pd.Series(result.resid).dropna()
106     smear = float(np.mean(np.exp(residuals))) if len(residuals) else 1.0
107
108     residual_pool = residuals.to_numpy(dtype=float)
109     residual_pool = residual_pool - np.mean(residual_pool)
110     rng = np.random.default_rng(SEED + int(train_end.strftime("%Y%m%d"))) + sum(ord(ch) for ch in str(full["category_name"].iloc[0])) + (0 if target == "net_sales_quantity_kg" else 10000)
111     n_simulations = bootstrap_repetitions

```

```

112 simulated_log = np.empty((n_simulations, len(forecast_dates)), dtype=float)
113 max_start = max(len(residual_pool) - len(forecast_dates), 0)
114 block_starts = rng.integers(0, max_start + 1, size=n_simulations)
115 innovations = np.vstack(
116     [residual_pool[start : start + len(forecast_dates)] for start in block_starts]
117 )
118 parameters = result.params
119 lag_parameters = {lag: float(parameters[f"target_lag_{lag}"]) for lag in DYNAMIC_LAGS}
120 nonlag_parameters = parameters.drop(index=[f"target_lag_{lag}" for lag in DYNAMIC_LAGS])
121 deterministic_path = np.empty(len(forecast_dates), dtype=float)
122 for step, date in enumerate(forecast_dates):
123     base_row = sm.add_constant(exog_future.loc[[date]], has_constant="add").reindex(
124         columns=nonlag_parameters.index
125     )
126     base = float(np.dot(base_row.iloc[0].to_numpy(dtype=float), nonlag_parameters.to_numpy(dtype=float)))
127     deterministic_value = base
128     simulated_value = np.full(n_simulations, base)
129     for lag, coefficient in lag_parameters.items():
130         if step - lag >= 0:
131             lag_deterministic = deterministic_path[step - lag]
132             lag_simulated = simulated_log[:, step - lag]
133         else:
134             known_value = float(transformed.get(date - pd.Timedelta(days=lag), np.nan))
135             lag_deterministic = known_value
136             lag_simulated = np.full(n_simulations, known_value)
137             deterministic_value += coefficient * lag_deterministic
138             simulated_value += coefficient * lag_simulated
139     deterministic_path[step] = deterministic_value
140     simulated_log[:, step] = simulated_value + innovations[:, step]
141 simulated_original = inverse_target(simulated_log, target)
142 point_forecast = simulated_original.mean(axis=0)
143 ci80 = np.quantile(simulated_original, [0.10, 0.90], axis=0)
144 ci95 = np.quantile(simulated_original, [0.025, 0.975], axis=0)
145 frame = pd.DataFrame(
146     {
147         "sale_date": forecast_dates,
148         "prediction": point_forecast,
149         "lower_80": ci80[0],
150         "upper_80": ci80[1],
151         "lower_95": ci95[0],
152         "upper_95": ci95[1],
153         "smearing_factor": smear,
154     }
155 )
156 return frame, result, scalings, ""

```

代码 3: 滚动样本外验证与逐品类选模

```

1 def run_validation(category_frame: pd.DataFrame) -> pd.DataFrame:
2     records: list[dict] = []
3     category = str(category_frame["category_name"].iloc[0])
4     for target in TARGETS:
5         for fold, origin in enumerate(VALIDATION_ORIGINS, start=1):
6             dates = pd.date_range(origin + pd.Timedelta(days=1), periods=7, freq="D")
7             actual_lookup = category_frame.set_index("sale_date")[target]
8             candidates: dict[str, pd.DataFrame] = {}
9             candidates["seasonal_median_8w"] = seasonal_median_forecast(
10                 category_frame, target, origin, dates
11             )
12             for name, specification in (
13                 ("dynamic_regression_calendar", "calendar"),
14                 ("dynamic_regression_state7", "state7"),
15             ):

```

```

16         if target == "weighted_wholesale_cost" and specification == "state7":
17             continue
18         forecast, result, _, warning_text = dynamic_regression_forecast(
19             category_frame, target, origin, dates, specification, bootstrap_repetitions=400
20         )
21         forecast["fit_converged"] = True
22         forecast["fit_warning"] = warning_text
23         candidates[name] = forecast
24     hgb, _, _ = hgb_forecast(category_frame, target, origin, dates)
25     candidates["hist_gradient_boosting"] = hgb
26     for model_name, forecast in candidates.items():
27         for _, row in forecast.iterrows():
28             date = pd.Timestamp(row["sale_date"])
29             records.append(
30                 {
31                     "target": target,
32                     "category_name": category,
33                     "fold": fold,
34                     "train_end": origin,
35                     "sale_date": date,
36                     "model": model_name,
37                     "actual": actual_lookup.get(date, np.nan),
38                     "prediction": row["prediction"],
39                     "lower_80": row.get("lower_80", np.nan),
40                     "upper_80": row.get("upper_80", np.nan),
41                     "lower_95": row.get("lower_95", np.nan),
42                     "upper_95": row.get("upper_95", np.nan),
43                     "fit_converged": row.get("fit_converged", True),
44                     "fit_warning": row.get("fit_warning", ""),
45                 }
46             )
47     return pd.DataFrame(records)
48
49
50 def aggregate_validation(validation: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
51     fold_rows: list[dict] = []
52     group_cols = ["target", "category_name", "model", "fold", "train_end"]
53     for keys, group in validation.groupby(group_cols, sort=False):
54         target, category, model, fold, train_end = keys
55         result = metrics(group["actual"], group["prediction"])
56         result.update(
57             {
58                 "target": target,
59                 "category_name": category,
60                 "model": model,
61                 "fold": fold,
62                 "train_end": train_end,
63                 "forecast_start": group["sale_date"].min(),
64                 "forecast_end": group["sale_date"].max(),
65                 "coverage_80": float(
66                     ((group["actual"] >= group["lower_80"]) & (group["actual"] <= group["upper_80"])).mean()
67                 )
68                 if group["lower_80"].notna().any()
69                 else np.nan,
70                 "coverage_95": float(
71                     ((group["actual"] >= group["lower_95"]) & (group["actual"] <= group["upper_95"])).mean()
72                 )
73                 if group["lower_95"].notna().any()
74                 else np.nan,
75                 "all_fits_converged": bool(group["fit_converged"].fillna(False).all()),
76             }
77         )
78     fold_rows.append(result)
79     fold_metrics = pd.DataFrame(fold_rows)

```

```

80
81 comparison_rows: list[dict] = []
82 for keys, group in validation.groupby(["target", "category_name", "model"], sort=False):
83     target, category, model = keys
84     result = metrics(group["actual"], group["prediction"])
85     result.update(
86         {
87             "target": target,
88             "category_name": category,
89             "model": model,
90             "n_folds": int(group["fold"].nunique()),
91             "validation_start": group["sale_date"].min(),
92             "validation_end": group["sale_date"].max(),
93             "coverage_80": float(
94                 ((group["actual"] >= group["lower_80"]) & (group["actual"] <= group["upper_80"])).mean()
95             )
96             if group["lower_80"].notna().any()
97             else np.nan,
98             "coverage_95": float(
99                 ((group["actual"] >= group["lower_95"]) & (group["actual"] <= group["upper_95"])).mean()
100             )
101             if group["lower_95"].notna().any()
102             else np.nan,
103             "all_fits_converged": bool(group["fit_converged"].fillna(False).all()),
104         }
105     )
106     comparison_rows.append(result)
107 comparison = pd.DataFrame(comparison_rows)
108 return fold_metrics, comparison
109
110
111 def select_models(comparison: pd.DataFrame) -> pd.DataFrame:
112     rows: list[dict] = []
113     for (target, category), group in comparison.groupby(["target", "category_name"], sort=False):
114         dynamic = group.loc[group["model"].str.startswith("dynamic_regression")].sort_values(["wape", "mae"])
115         primary = dynamic.iloc[0]
116         champion = group.sort_values(["wape", "mae"]).iloc[0]
117         baseline = group.loc[group["model"].eq("seasonal_median_8w")].iloc[0]
118         if baseline["wape"] < primary["wape"]:
119             operational_model = "seasonal_median_8w"
120             operational_wape = float(baseline["wape"])
121             operational_reason = "简单基线的滚动WAPE低于动态回归，按复杂度最小原则采用基线"
122         else:
123             operational_model = str(primary["model"])
124             operational_wape = float(primary["wape"])
125             operational_reason = "动态回归的滚动WAPE低于简单基线，且能提供参数解释和递归区间"
126         rows.append(
127             {
128                 "target": target,
129                 "category_name": category,
130                 "primary_model": primary["model"],
131                 "primary_wape": primary["wape"],
132                 "primary_mae": primary["mae"],
133                 "best_validation_model": champion["model"],
134                 "best_validation_wape": champion["wape"],
135                 "baseline_wape": baseline["wape"],
136                 "recommended_operational_model": operational_model,
137                 "recommended_operational_wape": operational_wape,
138                 "operational_selection_reason": operational_reason,
139                 "primary_wape_improvement_vs_baseline": (
140                     (baseline["wape"] - primary["wape"]) / baseline["wape"]
141                     if baseline["wape"] > 0
142                     else np.nan
143                 ),

```

```

144         "selection_rule": "主模型限定为可解释动态回归族，族内按四个七日滚动窗口WAPE最小选择；同时另报全候选冠军",
145     }
146 )
147 return pd.DataFrame(rows)

```

C.3 价格响应与随机情景

代码 4 在单品日面板上进行双向固定效应残差化并估计品类价格响应；代码 5 对需求与成本联合残差实施移动块抽样，形成优化所用的随机情景。

代码 4: 双向固定效应价格响应估计

```

1 def fit_price_response(item_daily: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
2     data = item_daily.copy()
3     data = data.loc[
4         data["net_sales_quantity_kg"].gt(0)
5         & data["weighted_sale_price"].gt(0)
6         & data["weighted_wholesale_cost"].gt(0)
7         & data["sale_transaction_count"].gt(0)
8     ].copy()
9     data["log_qty"] = np.log1p(data["net_sales_quantity_kg"].astype(float))
10    data["log_price_cost"] = np.log(
11        data["weighted_sale_price"].astype(float)
12        / data["weighted_wholesale_cost"].astype(float)
13    )
14    data["log_cost"] = np.log(data["weighted_wholesale_cost"].astype(float))
15    data["discount_share"] = (
16        data["discount_sale_count"].astype(float)
17        / data["sale_transaction_count"].astype(float)
18    ).clip(0, 1)
19    end = data["sale_date"].max()
20    windows = [(("全样本", None), ("近730日", 730), ("近365日", 365))]
21    rows: list[dict[str, object]] = []
22    for category in CATEGORY_ORDER:
23        category_data = data.loc[data["category_name"].eq(category)].copy()
24        for window_name, days in windows:
25            group = category_data.copy()
26            if days is not None:
27                group = group.loc[group["sale_date"].ge(end - pd.Timedelta(days=days - 1))].copy()
28            for variable in ["log_price_cost", "log_cost"]:
29                lower, upper = group[variable].quantile([0.01, 0.99])
30                group[variable] = group[variable].clip(lower, upper)
31            columns = ["log_qty", "log_price_cost", "log_cost", "discount_share"]
32            demeaned = alternating_two_way_demean(
33                group, columns, group_a="item_code", group_b="sale_date"
34            )
35            x = demeaned[["log_price_cost", "log_cost", "discount_share"]]
36            y = demeaned["log_qty"]
37            nonconstant = x.std(ddof=0).gt(1e-10)
38            x = x.loc[:, nonconstant]
39            fit = sm.OLS(y, x).fit(
40                cov_type="cluster",
41                cov_kws={"groups": group["item_code"].astype(str), "use_correction": True},
42            )
43            beta = float(fit.params.get("log_price_cost", np.nan))
44            se = float(fit.bse.get("log_price_cost", np.nan))
45            pvalue = float(fit.pvalues.get("log_price_cost", np.nan))
46            rows.append(
47                {
48                    "category_name": category,

```

```

49         "window": window_name,
50         "window_days": days,
51         "n_item_days": int(len(group)),
52         "n_items": int(group["item_code"].nunique()),
53         "n_dates": int(group["sale_date"].nunique()),
54         "beta_log_price_cost": beta,
55         "cluster_se": se,
56         "ci95_lower": beta - 1.96 * se,
57         "ci95_upper": beta + 1.96 * se,
58         "p_value": pvalue,
59         "within_r_squared": float(fit.rsquared),
60         "estimator": "双向固定效应组内OLS: 按单品聚类稳健标准误",
61         "interpretation": "历史条件关联, 不作因果价格弹性解释",
62     }
63 )
64 estimates = pd.DataFrame(rows)
65 summary_rows: list[dict[str, object]] = []
66 for category in CATEGORY_ORDER:
67     group = estimates.loc[estimates["category_name"].eq(category)].set_index("window")
68     betas = group["beta_log_price_cost"].astype(float)
69     full_p = float(group.loc["全样本", "p_value"])
70     stable = bool((betas < 0).all() and full_p <= 0.10)
71     negative = betas.loc[betas < 0]
72     if len(negative) == 0:
73         weak = medium = strong = 0.0
74         status = "三个窗口均无负向关联, 优化中不调整需求"
75     else:
76         empirical = float(np.median(negative))
77         if stable:
78             weak = 0.5 * empirical
79             medium = empirical
80             strong = 1.5 * empirical
81             status = "三窗口稳定负向; 以经验中位数构造三档响应"
82         else:
83             weak = 0.0
84             medium = 0.5 * empirical
85             strong = empirical
86             status = "方向或显著性不稳定; 弱情景设为零响应"
87     weak, medium, strong = [float(np.clip(x, -3.0, 0.0)) for x in (weak, medium, strong)]
88     summary_rows.append(
89         {
90             "category_name": category,
91             "beta_full": float(group.loc["全样本", "beta_log_price_cost"]),
92             "p_full": full_p,
93             "beta_730": float(group.loc["近730日", "beta_log_price_cost"]),
94             "beta_365": float(group.loc["近365日", "beta_log_price_cost"]),
95             "all_windows_negative": bool((betas < 0).all()),
96             "stable_negative_for_scenarios": stable,
97             "beta_weak": weak,
98             "beta_medium": medium,
99             "beta_strong": strong,
100             "scenario_status": status,
101         }
102     )
103 return estimates, pd.DataFrame(summary_rows)

```

代码 5: 移动块 Bootstrap 随机情景生成

```

1 def build_joint_scenarios(
2     forecast: pd.DataFrame,
3     rolling: pd.DataFrame,
4     selection: pd.DataFrame,
5     beta_summary: pd.DataFrame,

```

```

6     n_scenarios: int = N_SCENARIOS,
7     block_length: int = HORIZON,
8 ) -> ScenarioSet:
9     dates = pd.date_range(FORECAST_DATE, periods=HORIZON, freq="D")
10    selected = selection.set_index(["target", "category_name"])["recommended_operational_model"]
11    residual_frames = []
12    for target in ["net_sales_quantity_kg", "weighted_wholesale_cost"]:
13        for category in CATEGORY_ORDER:
14            model = selected.loc[(target, category)]
15            group = rolling.loc[
16                rolling["target"].eq(target)
17                & rolling["category_name"].eq(category)
18                & rolling["model"].eq(model)
19            ].sort_values("sale_date")
20            if target == "net_sales_quantity_kg":
21                resid = np.log1p(group["actual"].clip(lower=0)) - np.log1p(
22                    group["prediction"].clip(lower=0)
23                )
24            else:
25                resid = np.log(group["actual"].clip(lower=1e-8)) - np.log(
26                    group["prediction"].clip(lower=1e-8)
27                )
28            frame = pd.DataFrame(
29                {
30                    "sale_date": group["sale_date"].to_numpy(),
31                    f"{target}|{category}": resid.to_numpy(),
32                }
33            )
34            residual_frames.append(frame)
35    joint = residual_frames[0]
36    for frame in residual_frames[1:]:
37        joint = joint.merge(frame, on="sale_date", how="inner", validate="one_to_one")
38    joint = joint.sort_values("sale_date").reset_index(drop=True)
39    value_columns = [c for c in joint.columns if c != "sale_date"]
40    for column in value_columns:
41        lower, upper = joint[column].quantile([0.025, 0.975])
42        joint[column] = joint[column].clip(lower, upper)
43    block_length = int(block_length)
44    if block_length < 1 or block_length > HORIZON:
45        raise ValueError(f"block_length必须位于1至{HORIZON}之间")
46    if len(joint) < max(HORIZON, block_length):
47        raise ValueError("联合滚动残差不足以构造预测期情景")
48    rng = np.random.default_rng(SEED)
49    blocks_per_scenario = int(math.ceil(HORIZON / block_length))
50    start_matrix = rng.integers(
51        0,
52        len(joint) - block_length + 1,
53        size=(n_scenarios, blocks_per_scenario),
54    )
55    blocks = np.stack(
56        [
57            np.concatenate(
58                [
59                    joint.iloc[start : start + block_length][value_columns].to_numpy()
60                    for start in starts_row
61                ],
62                axis=0,
63            )[ :HORIZON]
64            for starts_row in start_matrix
65        ]
66    )
67    starts = start_matrix[:, 0]
68    column_index = {column: idx for idx, column in enumerate(value_columns)}
69    beta_tier = np.tile(np.arange(3), math.ceil(n_scenarios / 3))[:n_scenarios]

```

```

70 rng.shuffle(beta_tier)
71 beta_lookup = beta_summary.set_index("category_name")
72 demand: dict[str, np.ndarray] = {}
73 cost: dict[str, np.ndarray] = {}
74 beta: dict[str, np.ndarray] = {}
75 for category in CATEGORY_ORDER:
76     d_point = (
77         forecast.loc[
78             forecast["target"].eq("net_sales_quantity_kg")
79             & forecast["category_name"].eq(category)
80         ]
81         .set_index("sale_date")
82         .reindex(dates)["prediction"]
83         .to_numpy(dtype=float)
84     )
85     c_point = (
86         forecast.loc[
87             forecast["target"].eq("weighted_wholesale_cost")
88             & forecast["category_name"].eq(category)
89         ]
90         .set_index("sale_date")
91         .reindex(dates)["prediction"]
92         .to_numpy(dtype=float)
93     )
94     d_resid = blocks[:, :, column_index[f"net_sales_quantity_kg|{category}"]]
95     c_resid = blocks[:, :, column_index[f"weighted_wholesale_cost|{category}"]]
96     demand[category] = np.maximum(np.exp(np.log1p(d_point)[None, :] + d_resid) - 1.0, 0.0)
97     cost[category] = np.maximum(c_point[None, :] * np.exp(c_resid), 0.01)
98     tier_values = beta_lookup.loc[category, ["beta_weak", "beta_medium", "beta_strong"]].to_numpy(dtype=float)
99     beta[category] = tier_values[beta_tier]
100 return ScenarioSet(dates, demand, cost, beta, starts, beta_tier)

```

C.4 问题二 Pareto 优化

代码 6 生成品类价格—补货选项并计算情景利润；代码 7 扫描最低覆盖水平、求解 MILP 并按最大弦距规则选择 Pareto 折中点。

代码 6: 问题二分档选项与 MILP 求解器

```

1 def solve_grouped_options(
2     options: pd.DataFrame,
3     group_columns: list[str],
4     service_level: float | None,
5     service_group: str,
6     profit_column: str,
7     demand_column: str,
8     sales_column: str,
9     count_column: str | None = None,
10    count_lower: int | None = None,
11    count_upper: int | None = None,
12    count_equal: int | None = None,
13 ) -> tuple[pd.DataFrame, dict[str, object]]:
14     options = options.reset_index(drop=True).copy()
15     n = len(options)
16     row_indices: list[int] = []
17     col_indices: list[int] = []
18     data: list[float] = []
19     lower: list[float] = []
20     upper: list[float] = []
21     row_id = 0

```

```

22 for _, group in options.groupby(group_columns, sort=False, observed=True):
23     for idx in group.index:
24         row_indices.append(row_id)
25         col_indices.append(int(idx))
26         data.append(1.0)
27     lower.append(1.0)
28     upper.append(1.0)
29     row_id += 1
30 if service_level is not None:
31     for _, group in options.groupby(service_group, sort=False, observed=True):
32         coefficients = group[sales_column].to_numpy(dtype=float) - service_level * group[
33             demand_column
34         ].to_numpy(dtype=float)
35         for idx, value in zip(group.index, coefficients):
36             row_indices.append(row_id)
37             col_indices.append(int(idx))
38             data.append(float(value))
39         lower.append(0.0)
40         upper.append(np.inf)
41         row_id += 1
42 if count_column is not None:
43     coefficients = options[count_column].to_numpy(dtype=float)
44     for idx, value in enumerate(coefficients):
45         if value != 0:
46             row_indices.append(row_id)
47             col_indices.append(idx)
48             data.append(float(value))
49     if count_equal is not None:
50         lower.append(float(count_equal))
51         upper.append(float(count_equal))
52     else:
53         lower.append(float(count_lower if count_lower is not None else -np.inf))
54         upper.append(float(count_upper if count_upper is not None else np.inf))
55     row_id += 1
56 matrix = coo_matrix((data, (row_indices, col_indices)), shape=(row_id, n)).tocsr()
57 result = milp(
58     c=options[profit_column].to_numpy(dtype=float),
59     integrality=np.ones(n, dtype=int),
60     bounds=Bounds(np.zeros(n), np.ones(n)),
61     constraints=LinearConstraint(matrix, np.asarray(lower), np.asarray(upper)),
62     options={"mip_rel_gap": 1e-7, "time_limit": 180},
63 )
64 raw_mip_gap = getattr(result, "mip_gap", np.nan)
65 mip_gap = float(raw_mip_gap) if raw_mip_gap is not None else np.nan
66 meta = {
67     "success": bool(result.success),
68     "status": int(result.status),
69     "message": str(result.message),
70     "objective": float(-result.fun) if result.fun is not None else np.nan,
71     "mip_gap": mip_gap,
72 }
73 if not result.success or result.x is None:
74     return options.iloc[0:0].copy(), meta
75 chosen = options.loc[result.x > 0.5].copy()
76 return chosen, meta
77
78
79 def q2_plan_from_options(options: pd.DataFrame, mode: str, service_level: float | None) -> tuple[pd.DataFrame, dict[str, object]]:
80     prefix = "deterministic" if mode == "deterministic" else "expected"
81     if service_level is None:
82         profit_col = f"{prefix}_profit_yuan"
83         chosen = (
84             options.sort_values(profit_col, ascending=False)

```

```

85     .groupby(["category_name", "sale_date"], observed=True, sort=False)
86     .head(1)
87     .copy()
88 )
89     return chosen, {"success": True, "status": 0, "message": "逐组期望利润最大化", "objective": float(chosen[profit_col].sum())}, {"mip_gap": 0.0}
90     return solve_grouped_options(
91         options,
92         group_columns=["category_name", "sale_date"],
93         service_level=service_level,
94         service_group="category_name",
95         profit_column=f"{prefix}_profit_yuan",
96         demand_column=f"{prefix}_demand_kg",
97         sales_column=f"{prefix}_sales_kg",
98     )

```

代码 7: 最大弦距、Pareto 前沿扫描与折中选择

```

1  def solve_maximin_options(
2      options: pd.DataFrame,
3      group_columns: list[str],
4      service_group: str,
5      demand_column: str,
6      fulfilled_column: str,
7      count_column: str | None = None,
8      count_lower: int | None = None,
9      count_upper: int | None = None,
10     count_equal: int | None = None,
11 ) -> tuple[pd.DataFrame, dict[str, object]]:
12     """Maximize the minimum bounded baseline-coverage rate."""
13     options = options.reset_index(drop=True).copy()
14     n_options = len(options)
15     lambda_index = n_options
16     row_indices: list[int] = []
17     col_indices: list[int] = []
18     values: list[float] = []
19     lower: list[float] = []
20     upper: list[float] = []
21     row_id = 0
22
23     for _, group in options.groupby(group_columns, sort=False, observed=True):
24         for idx in group.index:
25             row_indices.append(row_id)
26             col_indices.append(int(idx))
27             values.append(1.0)
28             lower.append(1.0)
29             upper.append(1.0)
30             row_id += 1
31
32     representative = (
33         options.groupby(group_columns, sort=False, observed=True)
34         .first()
35         .reset_index()
36     )
37     denominators = representative.groupby(service_group, observed=True)[demand_column].sum()
38     for group_name, group in options.groupby(service_group, sort=False, observed=True):
39         for idx, value in zip(group.index, group[fulfilled_column].to_numpy(dtype=float)):
40             if value != 0:
41                 row_indices.append(row_id)
42                 col_indices.append(int(idx))
43                 values.append(float(value))
44             row_indices.append(row_id)
45             col_indices.append(lambda_index)

```

```

46     values.append(-float(denominators.loc[group_name]))
47     lower.append(0.0)
48     upper.append(np.inf)
49     row_id += 1
50
51 if count_column is not None:
52     for idx, value in enumerate(options[count_column].to_numpy(dtype=float)):
53         if value != 0:
54             row_indices.append(row_id)
55             col_indices.append(idx)
56             values.append(float(value))
57         if count_equal is not None:
58             lower.append(float(count_equal))
59             upper.append(float(count_equal))
60         else:
61             lower.append(float(count_lower if count_lower is not None else -np.inf))
62             upper.append(float(count_upper if count_upper is not None else np.inf))
63     row_id += 1
64
65 matrix = coo_matrix(
66     (values, (row_indices, col_indices)), shape=(row_id, n_options + 1)
67 ).tocsr()
68 objective = np.zeros(n_options + 1)
69 objective[lamba_index] = -1.0
70 integrality = np.r_[np.ones(n_options, dtype=int), 0]
71 result = milp(
72     c=objective,
73     integrality=integrality,
74     bounds=Bounds(np.zeros(n_options + 1), np.ones(n_options + 1)),
75     constraints=LinearConstraint(matrix, np.asarray(lower), np.asarray(upper)),
76     options={"mip_rel_gap": 1e-8, "time_limit": 240},
77 )
78 raw_gap = getattr(result, "mip_gap", np.nan)
79 meta = {
80     "success": bool(result.success),
81     "status": int(result.status),
82     "message": str(result.message),
83     "max_min_coverage": (
84         float(result.x[lamba_index]) if result.success and result.x is not None else np.nan
85     ),
86     "mip_gap": float(raw_gap) if raw_gap is not None else np.nan,
87 }
88 if not result.success or result.x is None:
89     return options.iloc[0:0].copy(), meta
90 return options.loc[result.x[:n_options] > 0.5].copy(), meta
91
92
93 def choose_pareto_knee(frontier: pd.DataFrame) -> str:
94     valid = frontier.loc[frontier["solver_success"]].copy()
95     valid = valid.sort_values(
96         ["min_category_baseline_coverage_rate", "expected_profit_yuan"],
97         ascending=[True, False],
98     )
99     nondominated = []
100     for idx, row in valid.iterrows():
101         dominated = (
102             (valid["min_category_baseline_coverage_rate"] >= row["min_category_baseline_coverage_rate"] - 1e-10)
103             & (valid["expected_profit_yuan"] >= row["expected_profit_yuan"] - 1e-8)
104             & (
105                 (valid["min_category_baseline_coverage_rate"] > row["min_category_baseline_coverage_rate"] + 1e-10)
106                 | (valid["expected_profit_yuan"] > row["expected_profit_yuan"] + 1e-8)
107             )
108         ).any()
109         if not dominated:

```

```

110     nondominated.append(idx)
111     curve = valid.loc[nondominated].sort_values("min_category_baseline_coverage_rate").copy()
112     frontier["is_nondominated"] = frontier.index.isin(curve.index)
113     frontier["knee_score"] = np.nan
114     if len(curve) == 1:
115         return str(curve.iloc[0]["solution_id"])
116
117     x = curve["min_category_baseline_coverage_rate"].to_numpy(dtype=float)
118     y = curve["expected_profit_yuan"].to_numpy(dtype=float)
119     x_norm = (x - x.min()) / max(x.max() - x.min(), 1e-12)
120     y_norm = (y - y.min()) / max(y.max() - y.min(), 1e-12)
121     x1, y1, x2, y2 = x_norm[0], y_norm[0], x_norm[-1], y_norm[-1]
122     denominator = max(np.hypot(y2 - y1, x2 - x1), 1e-12)
123     distances = np.abs((y2 - y1) * x_norm - (x2 - x1) * y_norm + x2 * y1 - y2 * x1) / denominator
124     if len(curve) > 2:
125         distances[0] = -np.inf
126         distances[-1] = -np.inf
127     if not np.isfinite(distances).any() or float(np.nanmax(distances)) <= 1e-9:
128         distances = -np.hypot(1.0 - x_norm, 1.0 - y_norm)
129     curve["knee_score"] = distances
130     frontier.loc[curve.index, "knee_score"] = curve["knee_score"]
131     return str(curve.loc[curve["knee_score"].idxmax(), "solution_id"])

```

C.5 问题三份额收缩与两阶段优化

代码 8 完成滚动交叉验证选择份额收缩强度、构造单品情景，并依次求解“最大化最低覆盖率”和“保持覆盖率最大化利润”两个阶段。

代码 8: 份额收缩交叉验证与问题三两阶段 MILP

```

1 def select_kappa_ratios(
2     item_daily: pd.DataFrame, candidates: pd.DataFrame
3 ) -> pd.DataFrame:
4     end = pd.Timestamp("2023-06-30")
5     origins = [end - pd.Timedelta(days=7 * week) for week in range(12, 0, -1)]
6     rows: list[dict[str, object]] = []
7     candidate_codes = set(candidates["item_code"].astype(str))
8     data = item_daily.loc[item_daily["item_code"].astype(str).isin(candidate_codes)].copy()
9     for category, group_items in candidates.groupby("category_name", observed=True):
10         codes = group_items["item_code"].astype(str).tolist()
11         scores = {float(ratio): [] for ratio in KAPPA_RATIOS}
12         weights = {float(ratio): [] for ratio in KAPPA_RATIOS}
13         for origin in origins:
14             recent = data.loc[
15                 data["category_name"].eq(category)
16                 & data["sale_date"].between(origin - pd.Timedelta(days=55), origin)
17             ].groupby("item_code", observed=True)["net_sales_quantity_kg"].sum().clip(lower=0)
18             long = data.loc[
19                 data["category_name"].eq(category)
20                 & data["sale_date"].between(origin - pd.Timedelta(days=364), origin)
21             ].groupby("item_code", observed=True)["net_sales_quantity_kg"].sum().clip(lower=0)
22             future = data.loc[
23                 data["category_name"].eq(category)
24                 & data["sale_date"].between(origin + pd.Timedelta(days=1), origin + pd.Timedelta(days=7))
25             ].groupby("item_code", observed=True)["net_sales_quantity_kg"].sum().clip(lower=0)
26             recent_v = recent.reindex(codes, fill_value=0.0).to_numpy(dtype=float)
27             long_v = long.reindex(codes, fill_value=0.0).to_numpy(dtype=float)
28             future_v = future.reindex(codes, fill_value=0.0).to_numpy(dtype=float)
29             if recent_v.sum() <= 0 or future_v.sum() <= 0:
30                 continue

```

```

31     prior = long_v / long_v.sum() if long_v.sum() > 0 else np.repeat(1 / len(codes), len(codes))
32     actual = future_v / future_v.sum()
33     for ratio in KAPPA_RATIOS:
34         strength = float(ratio * recent_v.sum())
35         predicted = (recent_v + strength * prior) / (recent_v.sum() + strength)
36         score = -float(np.sum(actual * np.log(np.clip(predicted, 1e-12, 1.0))))
37         scores[float(ratio)].append(score)
38         weights[float(ratio)].append(float(future_v.sum()))
39     for ratio in KAPPA_RATIOS:
40         ratio = float(ratio)
41         mean_score = (
42             float(np.average(scores[ratio], weights=weights[ratio]))
43             if scores[ratio]
44             else np.nan
45         )
46         rows.append(
47             {
48                 "category_name": category,
49                 "kappa_ratio": ratio,
50                 "weighted_cross_entropy": mean_score,
51                 "validation_origins": len(scores[ratio]),
52             }
53         )
54     result = pd.DataFrame(rows)
55     result["selected"] = False
56     for category, group in result.groupby("category_name", observed=True):
57         valid = group.dropna(subset=["weighted_cross_entropy"])
58         if valid.empty:
59             idx = group.loc[np.isclose(group["kappa_ratio"], 0.20)].index[0]
60         else:
61             idx = valid["weighted_cross_entropy"].idxmin()
62         result.loc[idx, "selected"] = True
63     return result
64
65
66 def candidate_share_table_cv(
67     item_daily: pd.DataFrame,
68     item_master: pd.DataFrame,
69     loss: pd.DataFrame,
70 ) -> tuple[pd.DataFrame, pd.DataFrame]:
71     candidates = base.candidate_share_table(item_daily, item_master, loss, share_window=56)
72     cv = select_kappa_ratios(item_daily, candidates)
73     selected = cv.loc[cv["selected"]].set_index("category_name")["kappa_ratio"]
74     updated = candidates.copy()
75     updated["kappa_ratio"] = updated["category_name"].map(selected).fillna(0.20)
76     shares = []
77     for _, group in updated.groupby("category_name", observed=True):
78         recent_total = float(group["recent_sales_kg"].sum())
79         long_total = float(group["long_sales_kg"].sum())
80         prior = (
81             group["long_sales_kg"] / long_total
82             if long_total > 0
83             else pd.Series(np.repeat(1.0 / len(group), len(group)), index=group.index)
84         )
85         strength = float(group["kappa_ratio"].iloc[0] * recent_total)
86         weight = (group["recent_sales_kg"] + strength * prior) / max(recent_total + strength, 1e-9)
87         shares.append(pd.Series(weight.to_numpy(), index=group.index))
88     updated["demand_share"] = pd.concat(shares).sort_index()
89     return updated, cv
90
91
92 def q2_category_summary(daily: pd.DataFrame) -> pd.DataFrame:
93     result = (
94         daily.groupby("category_name", observed=True)

```

```

95     .agg(
96         order_qty_kg=("order_qty_kg", "sum"),
97         baseline_expected_demand_kg=("baseline_expected_demand_kg", "sum"),
98         expected_demand_kg=("expected_demand_kg", "sum"),
99         expected_sales_kg=("expected_sales_kg", "sum"),
100         expected_baseline_fulfilled_kg=("expected_baseline_fulfilled_kg", "sum"),
101         expected_profit_yuan=("expected_profit_yuan", "sum"),
102         expected_waste_kg=("expected_waste_kg", "sum"),
103     )
104     .reindex(base.CATEGORY_ORDER)
105     .reset_index()
106 )
107 result["service_rate"] = result["expected_sales_kg"] / result["expected_demand_kg"]
108 result["baseline_coverage_rate"] = (
109     result["expected_baseline_fulfilled_kg"] / result["baseline_expected_demand_kg"]
110 )
111 return result
112
113
114 def q3_category_summary(items: pd.DataFrame) -> pd.DataFrame:
115     result = (
116         items.groupby("category_name", observed=True)
117         .agg(
118             selected_sku_count=("selected", "sum"),
119             order_qty_kg=("order_qty_kg", "sum"),
120             baseline_expected_demand_kg=("baseline_expected_demand_kg", "sum"),
121             expected_demand_kg=("expected_demand_kg", "sum"),
122             expected_sales_kg=("expected_sales_kg", "sum"),
123             expected_baseline_fulfilled_kg=("expected_baseline_fulfilled_kg", "sum"),
124             expected_profit_yuan=("expected_profit_yuan", "sum"),
125             expected_waste_kg=("expected_waste_kg", "sum"),
126         )
127         .reindex(base.CATEGORY_ORDER)
128         .reset_index()
129     )
130     result["service_rate"] = result["expected_sales_kg"] / result["expected_demand_kg"]
131     result["baseline_coverage_rate"] = (
132         result["expected_baseline_fulfilled_kg"] / result["baseline_expected_demand_kg"]
133     )
134     return result
135
136
137 def solve_q3_lexicographic(
138     options: pd.DataFrame, count_equal: int | None = None
139 ) -> tuple[pd.DataFrame, dict[str, object]]:
140     _, stage1 = solve_maximin_options(
141         options,
142         group_columns=["item_code"],
143         service_group="category_name",
144         demand_column="baseline_expected_demand_kg",
145         fulfilled_column="expected_baseline_fulfilled_kg",
146         count_column="selected",
147         count_lower=27,
148         count_upper=33,
149         count_equal=count_equal,
150     )
151     if not stage1["success"]:
152         return options.iloc[0:0].copy(), {"success": False, "stage1": stage1}
153     floor = max(0.0, float(stage1["max_min_coverage"]) - NUMERICAL_TOLERANCE)
154     chosen, stage2 = base.solve_grouped_options(
155         options,
156         group_columns=["item_code"],
157         service_level=floor,
158         service_group="category_name",

```

```

159     profit_column="expected_profit_yuan",
160     demand_column="baseline_expected_demand_kg",
161     sales_column="expected_baseline_fulfilled_kg",
162     count_column="selected",
163     count_lower=27,
164     count_upper=33,
165     count_equal=count_equal,
166 )
167 return chosen, {
168     "success": bool(stage2["success"]),
169     "max_min_coverage": float(stage1["max_min_coverage"]),
170     "coverage_floor_stage2": floor,
171     "stage1_mip_gap": stage1.get("mip_gap", np.nan),
172     "stage2_mip_gap": stage2.get("mip_gap", np.nan),
173     "stage1_message": stage1.get("message", ""),
174     "stage2_message": stage2.get("message", ""),
175 }

```

C.6 显著性检验与稳健性分析

代码 9 实现滞后及残差相关的 HAC、FDR 推断和单品活跃度风险统计；代码 10 实现移动块长度、份额收缩强度和候选窗口敏感性检验。

代码 9: 问题一相关显著性与活跃度风险检验

```

1 def standardized_hac_association(
2     y: pd.Series,
3     x: pd.Series,
4     maxlags: int = 28,
5     rank: bool = False,
6 ) -> dict[str, float]:
7     pair = pd.concat([y.rename("y"), x.rename("x")], axis=1).dropna()
8     if rank:
9         pair = pair.rank(method="average")
10    if len(pair) < 30 or pair["x"].std(ddof=0) <= 0 or pair["y"].std(ddof=0) <= 0:
11        return {"n": len(pair), "coefficient": np.nan, "ci_low": np.nan, "ci_high": np.nan, "p_value": np.nan}
12    pair = (pair - pair.mean()) / pair.std(ddof=0)
13    design = sm.add_constant(pair[["x"]], has_constant="add")
14    fit = sm.OLS(pair["y"], design).fit(cov_type="HAC", cov_kws={"maxlags": maxlags})
15    interval = fit.conf_int(alpha=0.05).loc["x"]
16    return {
17        "n": int(len(pair)),
18        "coefficient": float(fit.params["x"]),
19        "ci_low": float(interval.iloc[0]),
20        "ci_high": float(interval.iloc[1]),
21        "p_value": float(fit.pvalues["x"]),
22    }
23
24
25 def effect_label(value: float) -> str:
26     magnitude = abs(float(value))
27     if magnitude >= 0.50:
28         return "强"
29     if magnitude >= 0.30:
30         return "中等"
31     return "弱"
32
33
34 def q1_inference(data: dict[str, pd.DataFrame]) -> tuple[pd.DataFrame, pd.DataFrame]:
35     daily = data["category_daily"].pivot(

```

```

36     index="sale_date", columns="category_name", values="net_sales_quantity_kg"
37 ).reindex(columns=base.CATEGORY_ORDER)
38
39 lag_rows: list[dict[str, object]] = []
40 for category in base.CATEGORY_ORDER:
41     series = daily[category]
42     for lag in LAGS:
43         result = standardized_hac_association(series, series.shift(lag), maxlags=28)
44         lag_rows.append({"category_name": category, "lag_days": lag, **result})
45 lag_frame = pd.DataFrame(lag_rows)
46 valid = lag_frame["p_value"].notna()
47 lag_frame["q_value"] = np.nan
48 lag_frame.loc[valid, "q_value"] = multipletests(
49     lag_frame.loc[valid, "p_value"], method="fdr_bh"
50 )[1]
51 lag_frame["effect_size"] = lag_frame["coefficient"].map(effect_label)
52 lag_frame["actionable"] = lag_frame["q_value"].lt(0.05) & lag_frame["coefficient"].abs().ge(0.30)
53
54 residual = pd.DataFrame(
55     {category: eda.residualize_calendar(daily[category]) for category in base.CATEGORY_ORDER}
56 )
57 correlation_rows: list[dict[str, object]] = []
58 for category_a, category_b in combinations(base.CATEGORY_ORDER, 2):
59     result = standardized_hac_association(
60         residual[category_a], residual[category_b], maxlags=28, rank=True
61     )
62     correlation_rows.append(
63         {"category_a": category_a, "category_b": category_b, **result}
64     )
65 correlation = pd.DataFrame(correlation_rows)
66 valid = correlation["p_value"].notna()
67 correlation["q_value"] = np.nan
68 correlation.loc[valid, "q_value"] = multipletests(
69     correlation.loc[valid, "p_value"], method="fdr_bh"
70 )[1]
71 correlation["effect_size"] = correlation["coefficient"].map(effect_label)
72 correlation["actionable"] = correlation["q_value"].lt(0.05) & correlation["coefficient"].abs().ge(0.30)
73 return lag_frame, correlation
74
75
76 def item_activity_risk(data: dict[str, pd.DataFrame]) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, float]]:
77     end = pd.Timestamp("2023-06-30")
78     item_daily = data["item_daily"].copy()
79     recent7_codes = set(
80         item_daily.loc[
81             item_daily["sale_date"].between(end - pd.Timedelta(days=6), end)
82             & item_daily["gross_sales_quantity_kg"].gt(0),
83             "item_code",
84         ].astype(str)
85     )
86     recent56 = item_daily.loc[
87         item_daily["item_code"].astype(str).isin(recent7_codes)
88         & item_daily["sale_date"].between(end - pd.Timedelta(days=55), end)
89     ].copy()
90     positive = recent56.loc[recent56["gross_sales_quantity_kg"].gt(0)].copy()
91     activity = (
92         positive.groupby(["item_code", "item_name", "category_name"], observed=True)
93         .agg(
94             active_days_56=("sale_date", "nunique"),
95             sales_kg_56=("gross_sales_quantity_kg", "sum"),
96             positive_day_mean_kg=("gross_sales_quantity_kg", "mean"),
97             positive_day_sd_kg=("gross_sales_quantity_kg", "std"),
98         )
99         .reset_index()

```

```

100 )
101 activity["positive_day_cv"] = (
102     activity["positive_day_sd_kg"] / activity["positive_day_mean_kg"]
103 ).fillna(0.0)
104 activity["average_calendar_day_sales_kg"] = activity["sales_kg_56"] / 56.0
105 activity["minimum_display_days"] = 2.5 / activity["average_calendar_day_sales_kg"].clip(lower=1e-9)
106 low_cut = float(activity["active_days_56"].quantile(1 / 3))
107 high_cut = float(activity["active_days_56"].quantile(2 / 3))
108 activity["activity_group"] = np.select(
109     [activity["active_days_56"].le(np.floor(low_cut)), activity["active_days_56"].ge(np.ceil(high_cut))],
110     ["低活跃", "高活跃"],
111     default="中活跃",
112 )
113 order = pd.CategoricalDtype(["低活跃", "中活跃", "高活跃"], ordered=True)
114 activity["activity_group"] = activity["activity_group"].astype(order)
115 summary = (
116     activity.groupby("activity_group", observed=True)
117     .agg(
118         item_count=("item_code", "size"),
119         active_days_median=("active_days_56", "median"),
120         positive_cv_median=("positive_day_cv", "median"),
121         minimum_display_days_median=("minimum_display_days", "median"),
122         minimum_display_days_q75=("minimum_display_days", lambda x: x.quantile(0.75)),
123     )
124     .reset_index()
125 )
126 thresholds = {"low_activity_max_days": float(np.floor(low_cut)), "high_activity_min_days": float(np.ceil(high_cut))}
127 return activity.sort_values(["activity_group", "active_days_56"]), summary, thresholds

```

代码 10: 问题二、三关键口径敏感性分析

```

1 def q2_block_sensitivity(
2     data: dict[str, pd.DataFrame], beta_summary: pd.DataFrame, policy: pd.DataFrame
3 ) -> pd.DataFrame:
4     rows: list[dict[str, object]] = []
5     plans: dict[int, pd.DataFrame] = {}
6     for block_length in BLOCK_LENGTHS:
7         scenarios = base.build_joint_scenarios(
8             data["forecast"], data["rolling"], data["selection"], beta_summary,
9             block_length=block_length,
10        )
11        options = base.q2_option_table(scenarios, data["forecast"], policy)
12        chosen, meta = base.solve_grouped_options(
13            options,
14            group_columns=["category_name", "sale_date"],
15            service_level=0.86,
16            service_group="category_name",
17            profit_column="expected_profit_yuan",
18            demand_column="baseline_expected_demand_kg",
19            sales_column="expected_baseline_fulfilled_kg",
20        )
21        if not meta["success"]:
22            raise RuntimeError(f"Q2块长{block_length}日方案求解失败: {meta}")
23        summary, daily, _ = base.evaluate_q2_plan(
24            chosen, scenarios, data["forecast"], policy
25        )
26        plans[block_length] = daily
27        rows.append(
28            {
29                "block_length_days": block_length,
30                "coverage_floor": 0.86,
31                **summary,
32            }

```

```

33     )
34     reference = plans[7].set_index(["category_name", "sale_date"])
35     for row in rows:
36         current = plans[int(row["block_length_days"])] .set_index(["category_name", "sale_date"])
37         aligned = current.join(reference[["order_qty_kg", "price_per_kg"]], rsuffix="_ref")
38         row["mean_absolute_order_change_kg_vs_7"] = float(
39             (aligned["order_qty_kg"] - aligned["order_qty_kg_ref"]).abs().mean()
40         )
41         row["same_price_share_vs_7"] = float(
42             np.isclose(aligned["price_per_kg"], aligned["price_per_kg_ref"], rtol=0, atol=1e-8).mean()
43         )
44     return pd.DataFrame(rows)
45
46
47 def apply_kappa_ratios(candidates: pd.DataFrame, ratios: dict[str, float]) -> pd.DataFrame:
48     updated = candidates.copy()
49     updated["kappa_ratio"] = updated["category_name"].map(ratios).astype(float)
50     share_parts = []
51     for _, group in updated.groupby("category_name", observed=True):
52         recent_total = float(group["recent_sales_kg"].sum())
53         long_total = float(group["long_sales_kg"].sum())
54         prior = (
55             group["long_sales_kg"] / long_total
56             if long_total > 0
57             else pd.Series(np.repeat(1.0 / len(group), len(group)), index=group.index)
58         )
59         strength = float(group["kappa_ratio"].iloc[0] * recent_total)
60         weights = (group["recent_sales_kg"] + strength * prior) / max(recent_total + strength, 1e-9)
61         share_parts.append(pd.Series(weights.to_numpy(), index=group.index))
62     updated["demand_share"] = pd.concat(share_parts).sort_index()
63     return updated
64
65
66 def candidates_with_cv(
67     data: dict[str, pd.DataFrame], candidate_window: int
68 ) -> tuple[pd.DataFrame, pd.DataFrame]:
69     candidates = base.candidate_share_table(
70         data["item_daily"], data["item_master"],
71         share_window=56, candidate_window=candidate_window,
72     )
73     cv = v2.select_kappa_ratios(data["item_daily"], candidates)
74     ratios = cv.loc[cv["selected"]].set_index("category_name")["kappa_ratio"].to_dict()
75     return apply_kappa_ratios(candidates, ratios), cv
76
77
78 def solve_q3_variant(
79     candidates: pd.DataFrame,
80     data: dict[str, pd.DataFrame],
81     q2_main: pd.DataFrame,
82     scenarios: base.ScenarioSet,
83 ) -> tuple[dict[str, float], set[str]]:
84     item_policy = base.item_policy_inputs(candidates, data["item_daily"], data["forecast"], q2_main)
85     item_policy = item_policy.merge(
86         candidates[["item_code", "kappa_ratio"]], on="item_code", how="left", validate="one_to_one"
87     )
88     options = base.q3_option_table(item_policy, q2_main, scenarios, data["forecast"])
89     chosen, meta = v2.solve_q3_lexicographic(options)
90     if not meta["success"]:
91         raise RuntimeError(f"Q3稳健性方案求解失败: {meta}")
92     summary, items, _ = base.evaluate_q3_plan(chosen, item_policy, q2_main, scenarios, data["forecast"])
93     codes = set(items.loc[items["selected"].eq(1), "item_code"].astype(str))
94     summary["stage1_max_min_coverage"] = float(meta["max_min_coverage"])
95     return summary, codes
96

```

```

97
98 def jaccard(left: set[str], right: set[str]) -> float:
99     return len(left & right) / max(len(left | right), 1)
100
101
102 def q3_sensitivity(
103     data: dict[str, pd.DataFrame], beta_summary: pd.DataFrame
104 ) -> tuple[pd.DataFrame, pd.DataFrame]:
105     scenarios = base.build_joint_scenarios(
106         data["forecast"], data["rolling"], data["selection"], beta_summary, block_length=7
107     )
108     q2_main = pd.read_csv(
109         ROOT / "output/gate_c_optimization_v2/q2_main_plan.csv", parse_dates=["sale_date"]
110     )
111     base_candidates, cv = candidates_with_cv(data, 7)
112     selected_ratios = cv.loc[cv["selected"]].set_index("category_name")["kappa_ratio"].to_dict()
113     ratio_maps = {
114         "no_shrinkage": {category: 0.0 for category in base.CATEGORY_ORDER},
115         "cross_validated": selected_ratios,
116         "strong_shrinkage": {category: 1.0 for category in base.CATEGORY_ORDER},
117     }
118     kappa_results: dict[str, tuple[dict[str, float], set[str]]] = {}
119     for mode in KAPPA_MODES:
120         candidates = apply_kappa_ratios(base_candidates, ratio_maps[mode])
121         kappa_results[mode] = solve_q3_variant(candidates, data, q2_main, scenarios)
122     reference_codes = kappa_results["cross_validated"][1]
123     kappa_rows = []
124     for mode in KAPPA_MODES:
125         summary, codes = kappa_results[mode]
126         kappa_rows.append(
127             {
128                 "kappa_mode": mode,
129                 "candidate_count": len(base_candidates),
130                 **summary,
131                 "selected_jaccard_vs_cv": jaccard(codes, reference_codes),
132             }
133         )
134
135     window_results: dict[int, tuple[dict[str, float], set[str], int]] = {}
136     for window in CANDIDATE_WINDOWS:
137         candidates, _ = candidates_with_cv(data, window)
138         summary, codes = solve_q3_variant(candidates, data, q2_main, scenarios)
139         window_results[window] = (summary, codes, len(candidates))
140     window_reference_codes = window_results[7][1]
141     window_rows = []
142     for window in CANDIDATE_WINDOWS:
143         summary, codes, count = window_results[window]
144         window_rows.append(
145             {
146                 "candidate_window_days": window,
147                 "candidate_count": count,
148                 **summary,
149                 "selected_jaccard_vs_7day": jaccard(codes, window_reference_codes),
150             }
151         )
152     return pd.DataFrame(kappa_rows), pd.DataFrame(window_rows)

```

关键输出分别保存在output/00_eda/、output/q2_forecast/、output/gate_c_optimization_v2/ 和output/supplementary_robustness/。约束回代结果显示：Q2 共有 42 个日期一品类策略，折中方案最低品类覆盖率为 0.860005；Q3 候选 49 个、入选 33 个，最低订购量 2.5 kg，第一、第二阶段最低覆盖率均为 0.936455；两问有界覆盖

率均未超过 1。